

Algorithm Engineering Notes

Academic Year 2026/2027

Contents

1	Algorithm Design, Computational Models, and Memory Hierarchy	1
1.1	Formal Definition of an Algorithm	1
1.1.1	Exact Algorithms, Artificial Intelligence, and Heuristics	2
1.2	Efficiency Measurement and Asymptotic Analysis	3
1.2.1	Why Not Count the Exact Number of Instructions? . .	3
1.2.2	Why Prefer Worst-Case Analysis?	3
1.3	Computational Models: RAM and Real Hardware	4
1.3.1	The Random Access Machine Model	4
1.3.2	Real Hardware: Memory Hierarchy and the Memory Wall	4
1.3.3	When the RAM Model Is Not Enough	6
1.4	Locality and the Two-Level Model	7
1.4.1	The Two-Level Memory Model	7
1.5	Worked Example: Linear Array Scan	9
1.5.1	Analysis in the Classical RAM Model	9
1.5.2	Analysis in the Two-Level Model	9
1.5.3	Linear Access versus Sparse Access	10
2	Strided Access, B-Trees, and the Impact of Virtual Paging	11
2.1	Non-Sequential Access: The $A_{s,b}$ Algorithm	11
2.1.1	Coprimality Condition	12
2.1.2	I/O Cost	12
2.2	Algorithmic Limits and Hardware Acceleration	13
2.3	Searching Ordered Sets: Binary Search versus B-Trees	14
2.3.1	Binary Search in Secondary Memory	14
2.3.2	B-Trees and B ⁺ -Trees	15
2.3.3	Quantitative Comparison	15
2.4	I/O-Aware Algorithms and Virtual Paging	16
2.4.1	Paging with an Oversized Working Set	16
2.4.2	Numerical Thrashing Example	17
2.5	Key Takeaways	17

3	Permutation, Sorting, and Multiway Merge Sort in External Memory	19
3.1	The Permutation vs. Sorting Problem	19
3.1.1	Problem Formulation	19
3.1.2	Naive Implementation on the RAM Model	21
3.1.3	Simulation Principle (RAM to Two-Level Memory)	21
3.1.4	The Relationship Between Sorting and Permutation	21
3.2	Permutation Complexity in External Memory	24
3.2.1	Corner Case Analysis	25
3.2.2	Numerical Verification on Realistic Architectural Parameters	25
3.3	External Memory Binary Merge Sort	26
3.3.1	Buffer Architecture in the Merge Procedure	26
3.3.2	Recursion Tree Complexity	27
3.3.3	Sub-optimality and Bottlenecks	28
3.4	Asymptotic Equivalence of the External Sorting Cost	29
3.5	Multiway Merge Sort (k -Way Merge Sort)	30
3.5.1	Internal Memory Configuration and Layout	30
3.5.2	Algorithm Execution and Tree Height Evolution	31
3.6	Optimizing Run Formation: The Snow Plow (<i>Replacement Selection</i>)	33
3.6.1	Operational Mechanism of Replacement Selection	33
3.6.2	Step-by-Step Execution Trace	34
3.6.3	Average Run Length Theorem	35
3.7	Parallelism Across Multiple Disks: <i>Disk Striping</i>	36
3.7.1	Parallel External Sorting Lower Bound	36
3.7.2	The Disk Striping Technique	36
3.7.3	Slowdown Analysis	37
3.7.4	Asymptotic Regimes and Physical Limitations	38
4	Lower Bounds for Sorting, Dual-Pivot Quicksort, and Bounded Quicksort	39
4.1	RAM Model: Lower Bounds and Decision Trees	39
4.1.1	The Decision Tree Model	40
4.1.2	The General Information-Theoretic Lower Bound	41
4.1.3	Fundamental Applications: Searching and Sorting	42
4.2	Two-Level Memory: Aggarwal-Vitter Lower Bound	43
4.2.1	The Nature of I/O and Free In-Memory Comparisons	43
4.2.2	Branching Factor (Fan-out) Calculation of the I/O Tree	44
4.2.3	Formal Derivation of the Aggarwal-Vitter Lower Bound	45
4.3	Quicksort: Duplicate Handling and Dual-Pivot Quicksort	47

4.3.1	The Equal Values Problem	48
4.3.2	3-Way Partitioning Algorithm and Numerical Execution Example	48
4.3.3	Architecture of Dual-Pivot Quicksort (Yaroslavskiy) . .	51
4.3.4	Invariants and Transition Logic	52
4.3.5	Execution Example and Trace of Dual-Pivot Quicksort	54
4.4	Bounded Quicksort (BQS) and Tail Recursion Elimination . .	55
4.4.1	Call Stack Vulnerability	56
4.4.2	Bounded Quicksort Algorithm (BQS)	56
4.4.3	Execution Example and Call Stack Management in Bounded Quicksort	58
4.4.4	Rigorous Analysis of the $\Theta(\log_2 n)$ Extra Space Bound	60
4.4.5	Hybrid Optimization with Insertion Sort Cutoff	61

Chapter 1

Algorithm Design, Computational Models, and Memory Hierarchy

Lecture 1

Introduction, computational models, and memory hierarchy

Date: September 15, 2026

1.1 Formal Definition of an Algorithm

An *algorithm* is a finite and ordered sequence of unambiguous, elementary instructions that, starting from a specified set of input data, produces a corresponding set of output data within a finite amount of time.

In the classical formulation by Donald Knuth in *The Art of Computer Programming* [1], an algorithm must satisfy five fundamental properties:

Finiteness. The algorithm must always halt after a finite number of computation steps for every admissible input configuration.

Definiteness. Each instruction must be stated clearly, precisely, and without ambiguity; the next execution state must be uniquely determined.

Input. The algorithm receives zero or more quantities specified from outside before execution begins.

Output. The algorithm produces one or more output quantities that have a specified relationship with the input data. The output must be meaningful for the problem being solved.

Effectiveness. Every prescribed operation must be elementary and physically executable in finite time using a bounded amount of resources, such as paper and pencil or a sequential machine.

1.1.1 Exact Algorithms, Artificial Intelligence, and Heuristics

A classical exact algorithm is distinguished from an AI or heuristic method by the guarantees it provides and by how it explores the space of decisions.

Table 1.1: Exact algorithms compared with AI and heuristic methods.

Dimension	Classical exact algorithm	AI or heuristic method
Correctness guarantee	Produces a correct solution for every admissible instance; optimality is guaranteed when the problem specification asks for an optimum.	May prioritize empirical quality or practical efficiency and may lack a universal guarantee for every instance.
Termination	Can be proved in advance using potential functions or loop variants.	Often tied to heuristic stopping criteria, such as a maximum number of epochs or gradient convergence.
Decision space	Determined by a precisely specified procedure, which may be deterministic or randomized.	Explored through heuristic search, statistical inference, or loss-function optimization.

Heuristic programming does not seek a formal proof of absolute optimality over every admissible instance. Instead, it seeks an efficient trade-off for computationally intractable problems, such as NP-complete or NP-hard problems.

1.2 Efficiency Measurement and Asymptotic Analysis

1.2.1 Why Not Count the Exact Number of Instructions?

Defining the exact running cost $T_A(n)$ as the actual number of clock cycles or elementary instructions presents several difficulties:

1. **Architecture dependence:** the result would depend on the specific CPU, instruction set architecture (ISA), and clock frequency.
2. **Toolchain dependence:** it would vary with the compiler, optimization level, and operating system.
3. **Lack of generality:** it would obscure the intrinsic quality of the algorithmic idea.

For this reason, algorithm analysis uses asymptotic notation. It abstracts away implementation details and multiplicative constants, and studies the growth rate of the cost function as the input size tends to infinity: $n \rightarrow \infty$.

Big- $\mathcal{O}$ (asymptotic upper bound):

$$f(n) = \mathcal{O}(g(n)) \iff \exists c > 0, n_0 > 0 \text{ such that} \\ 0 \leq f(n) \leq c g(n), \quad \forall n \geq n_0.$$

Big- Ω (asymptotic lower bound):

$$f(n) = \Omega(g(n)) \iff \exists c > 0, n_0 > 0 \text{ such that} \\ 0 \leq c g(n) \leq f(n), \quad \forall n \geq n_0.$$

Big- Θ (tight asymptotic bound):

$$f(n) = \Theta(g(n)) \iff f(n) = \mathcal{O}(g(n)) \wedge f(n) = \Omega(g(n)).$$

1.2.2 Why Prefer Worst-Case Analysis?

- **Absolute guarantee:** an upper bound guarantees that the algorithm will never take more time than the theoretical bound. This is essential for real-time and mission-critical systems.
- **Simple formal derivation:** worst-case analysis does not require probabilistic assumptions about the input distribution, which are often needed for average-case analysis.

1.3 Computational Models: RAM and Real Hardware

An algorithm is not executed in an abstract vacuum: it runs on a machine with processors, memories, and data-transfer mechanisms. A computational model specifies which machine details are relevant to the analysis and which are deliberately ignored. The RAM model is the standard starting point; real hardware then explains when that abstraction becomes insufficient.

1.3.1 The Random Access Machine Model

The Random Access Machine (RAM) model is a sequential, word-based abstraction inspired by the von Neumann architecture. A RAM consists of a processor, a finite program, and an addressable memory of words. Its usual assumptions are:

- **Sequential execution:** one processor executes one elementary instruction at a time.
- **Word-sized data:** arithmetic, comparison, assignment, and address calculation on a constant number of words take constant time.
- **Uniform memory access:** reading or writing any address costs $\mathcal{O}(1)$, independently of the address and of the access history.
- **Sufficient memory:** the input and the data structures fit in the modeled address space; the cost of allocating the space is analyzed separately when it matters.

The RAM model is useful because it makes algorithmic ideas comparable. For example, a loop that examines n array entries performs $\Theta(n)$ RAM operations, regardless of the particular processor used to run it. The model does not claim that a real machine has infinite memory or truly constant latency; it states which costs are being abstracted away.

1.3.2 Real Hardware: Memory Hierarchy and the Memory Wall

On a modern machine, the processor is much faster than the memories that hold large datasets. This growing gap between computation and data movement is known as the *memory wall*. Moreover, local memory is not a

single uniform resource: it is organized into levels with different capacities, latencies, and transfer granularities.

A simplified hierarchy, ordered from the fastest and smallest level to the slowest and largest one, is:

CPU registers $\longrightarrow$ L1 cache $\longrightarrow$ L2 cache $\longrightarrow$ L3 cache $\longrightarrow$ DRAM
 $\longrightarrow$ NVMe/SSD/disk $\longrightarrow$ remote storage / cloud.

Registers and caches are part of the processor's fast memory system; DRAM is the usual main memory; SSDs and disks provide persistent storage. Remote storage and cloud services are included here only as a broader data-access hierarchy, because they are reached through a network rather than through the local memory bus.

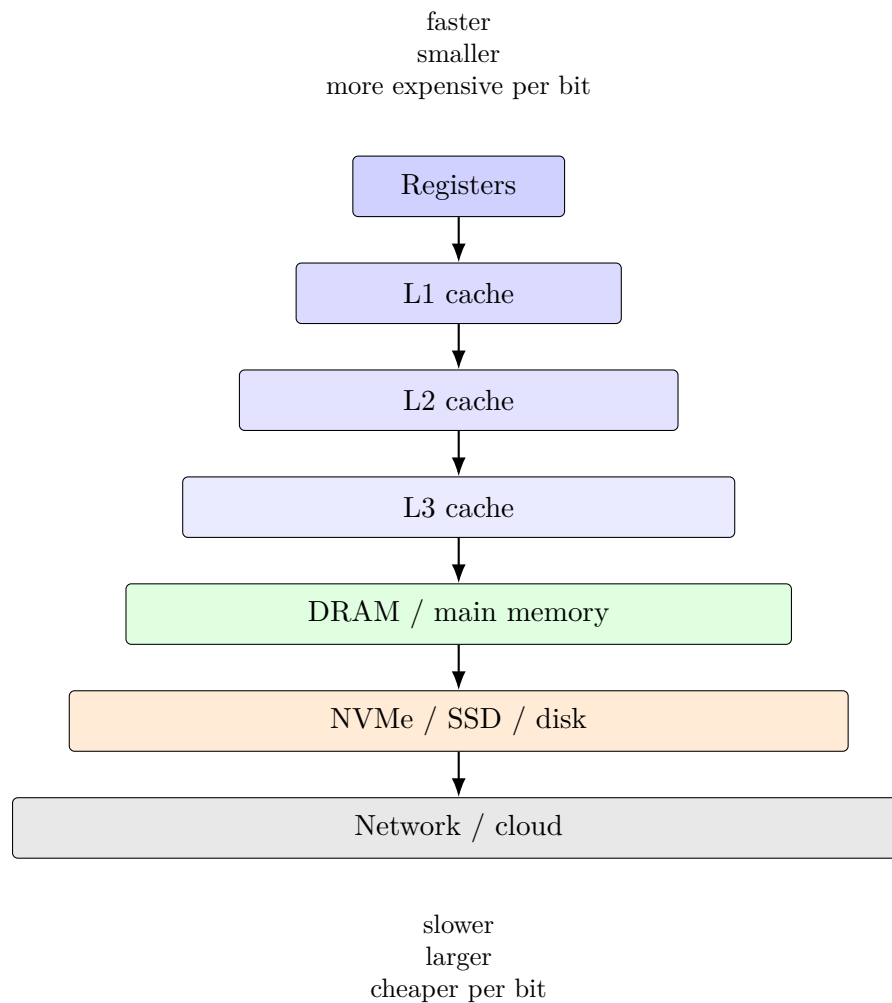


Figure 1.1: A hierarchical view of memory and storage.

Figure 1.1 describes the main local-memory levels: each lower layer trades lower access speed for greater capacity.

Moving downward in Figure 1.1, capacity generally grows while latency and transfer cost grow as well. The exact ratios depend on the hardware and workload, so values such as 100–200 times for DRAM relative to L1, or 10^4 – 10^6 times for disk relative to L1, should be read as order-of-magnitude examples rather than universal constants.

1.3.3 When the RAM Model Is Not Enough

The RAM model is a good first approximation when the computation is mostly sequential, the working set fits in main memory, and the cost of data

movement is not the dominant factor. A different model is preferable when one of the following effects controls performance:

GPU computing. SIMT (Single Instruction, Multiple Threads) execution uses many processing cores and distinguishes shared, local, and global memory spaces.

External memory. In out-of-core computation, the dataset does not fit in fast memory, so block transfers and I/O operations dominate the cost.

Parallel and distributed computation. Several processors or machines execute simultaneously, and synchronization and communication become explicit costs.

Quantum computing. Qubits, superposition, entanglement, and unitary gates require a model based on quantum operations rather than RAM words.

The engineering lesson is therefore not to discard the RAM model, but to use the simplest model that still captures the bottleneck of the target system.

1.4 Locality and the Two-Level Model

Modern systems exploit two statistical properties of real programs:

1. **Temporal locality:** if a memory location is referenced at time t , it is likely to be referenced again soon, at $t + \Delta t$.
2. **Spatial locality:** if a memory location x is referenced, nearby locations such as $x + 1$, $x + 2$, and so on are likely to be referenced soon.

1.4.1 The Two-Level Memory Model

When processing gigabytes or terabytes of data, the multilevel hierarchy can be abstracted as a two-level model, associated with the external-memory model of Aggarwal and Vitter [2], also known as the Disk Access Machine model.

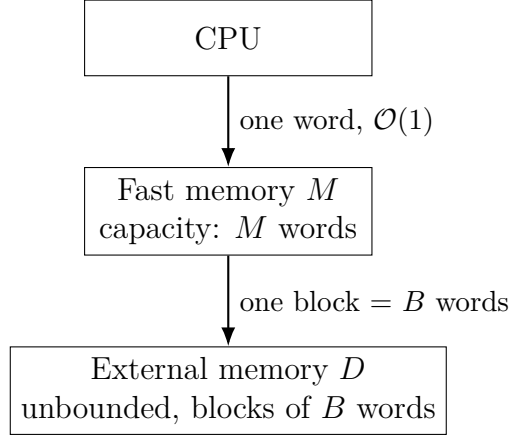


Figure 1.2: Two-level memory model with fast memory M and external memory D .

Figure 1.2 describes the two-level abstraction: the CPU operates on words in fast memory M , while transfers between M and the external memory D occur in blocks of B words.

Model Parameters

N : total number of problem elements to process.

M : size of internal fast memory, measured as the maximum number of elements that can be held simultaneously.

B : size of the block transferred in one I/O operation, with $M \geq 2B$.

Cost Metrics

1. **I/O complexity (primary metric)**: number of transfers of blocks of size B between slow and fast memory.
2. **CPU time (secondary metric)**: number of logical and arithmetic operations performed after data have been loaded into M .
3. **Space**: number of blocks allocated on secondary storage.

This abstraction is fractal: it can model both RAM–disk transfers and L3-cache–RAM transfers by redefining the meaning of M and B .

1.5 Worked Example: Linear Array Scan

Consider a contiguous array of N integers scanned in sequence, for example to compute the sum $\sum_{i=1}^N A[i]$.

Listing 1.1: Sequential array scan

```
1 long sum = 0;
2 for (int i = 0; i < N; i++) {
3     sum += A[i];
4 }
```

1.5.1 Analysis in the Classical RAM Model

- Memory accesses: N .
- Addition instructions: N .
- Asymptotic computation time: $\Theta(N)$.

1.5.2 Analysis in the Two-Level Model

Assume that the secondary memory stores array A consecutively in blocks of size B and that $A[0]$ is aligned with a block boundary. Without this alignment assumption, the scan may require at most one additional block transfer, without changing the asymptotic bound:

1. At the first access to $A[0]$, a miss occurs. The system transfers the block containing $A[0], \dots, A[B-1]$ into fast memory with one I/O.
2. The following $B-1$ accesses find their data already in fast memory, requiring no additional I/O. This exploits spatial locality.
3. A new I/O transfer is needed only when the index crosses a block boundary, that is, once every B elements.

Therefore, the total I/O cost of a linear scan is

$$\text{Scan}(N) = \left\lceil \frac{N}{B} \right\rceil = \Theta\left(\frac{N}{B}\right) \text{ I/O operations.}$$

1.5.3 Linear Access versus Sparse Access

- **Sequential access (high spatial locality):** reading N elements requires N/B I/O operations. Since B typically ranges from 64 bytes for a cache line to hundreds of kilobytes or megabytes for pages and disk blocks, the speedup over the naive model can be a factor of approximately B .
- **Random access or stride $k > B$ (no spatial locality):** every requested element may reside in a different block, causing N independent I/O operations. The cost degrades to $\Theta(N)$ I/Os, which is asymptotically slower by a factor of B .

Chapter 2

Strided Access, B-Trees, and the Impact of Virtual Paging

Lecture 2

Strided access, binary search versus B-Trees, and virtual paging

Date: September 16, 2026

This chapter studies how an algorithm's access pattern interacts with the transfer granularity of a memory system. The same arithmetic work can have very different costs depending on whether accesses preserve spatial locality, match the block size, or exceed the capacity of fast memory.

2.1 Non-Sequential Access: The $A_{s,b}$ Algorithm

Let S be an array containing n elements. The storage system transfers data in physical blocks of size B (a disk block or cache line). We also define a logical block size b , where $b \leq B$ and b divides B . The corresponding numbers of logical and physical blocks are

$$N' = \frac{n}{b}, \quad N_{\text{phys}} = \frac{n}{B}.$$

The algorithm $A_{s,b}$ computes the sum of the elements by visiting logical blocks in a stride pattern. The logical block visited at step j is

$$\text{block}(j) = (js) \bmod N', \quad j = 0, 1, \dots, N' - 1.$$

2.1.1 Coprimality Condition

The sequence visits every logical block exactly once if and only if

$$\gcd\left(s, \frac{n}{b}\right) = 1.$$

In that case, multiplication by s is a permutation of the residues modulo N' . If the greatest common divisor is $d > 1$, the sequence repeats after $N'/d = n/(bd)$ distinct logical blocks and therefore covers only a fraction of the array.

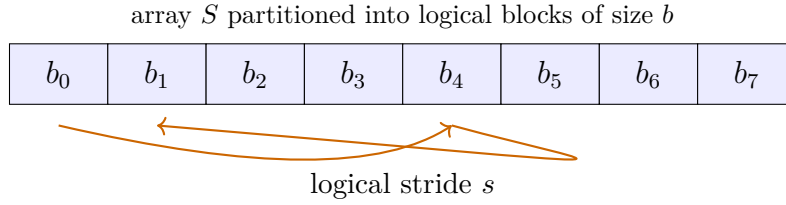


Figure 2.1: A modular stride visits logical blocks in a non-sequential order.

Figure 2.1 illustrates the logical access order. The physical cost depends on how this order maps to blocks of size B , rather than only on the number of arithmetic operations.

2.1.2 I/O Cost

A useful first approximation compares the physical distance between successive logical accesses, sb , with the physical block size B .

Table 2.1: Approximate I/O behavior of the strided access pattern.

Condition	Locality and behavior	I/O cost
$sb < B$	Several logical accesses can share one physical block; partial spatial locality is preserved.	$\Theta(n/B)$
$sb \geq B$	Successive accesses typically fall on different physical blocks. If $M < n$, the working set cannot remain entirely in fast memory.	$\Theta(n/b)$
$b = 1$ and $s \geq B$	Each logical access is one element and the stride destroys the benefit of block transfers.	$\Theta(n)$

The bounds in Table 2.1 describe the usual worst-case or capacity-miss regime. Exact constants depend on alignment, replacement policy, and the interaction between the modular sequence and physical blocks. In the last case, the cost is a factor of roughly B worse than a linear scan, which can exploit every transferred element.

2.2 Algorithmic Limits and Hardware Acceleration

Fix a maximum admissible execution time T_0 and assume that hardware or parallelism accelerates the computation by a factor K . The accelerated machine can therefore afford an original running time of

$$\frac{T(n)}{K} = T_0 \quad \Longleftrightarrow \quad T(n) = KT_0.$$

The effect on the largest solvable input depends on the growth rate of T .

Table 2.2: Effect of a K -fold hardware acceleration on solvable input size.

Algorithm	Complexity	Base hardware	Accelerated hardware	Effective gain
A_1	cn	$n_1 = T_0/c$	$n'_1 = Kn_1$	Multiplicative, K
A_2	cn^2	$n_2 = \sqrt{T_0/c}$	$n'_2 = \sqrt{K} n_2$	Sublinear, $\sqrt{K}$
A_3	$c2^n$	$n_3 = \log_2(T_0/c)$	$n'_3 = n_3 + \log_2 K$	Additive, $+\log_2 K$

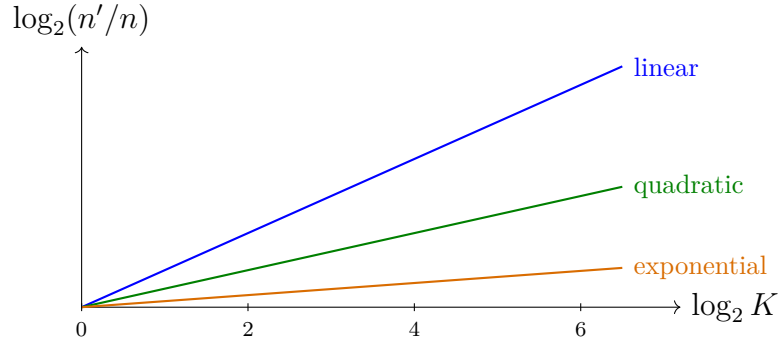


Figure 2.2: Relative benefit of hardware acceleration for representative growth rates.

Figure 2.2 shows why hardware improvements do not remove an asymptotically inefficient algorithm. For an exponential algorithm, even a 1000-fold speedup increases the solvable input size by only $\log_2(1000) \approx 10$ elements.

2.3 Searching Ordered Sets: Binary Search versus B-Trees

2.3.1 Binary Search in Secondary Memory

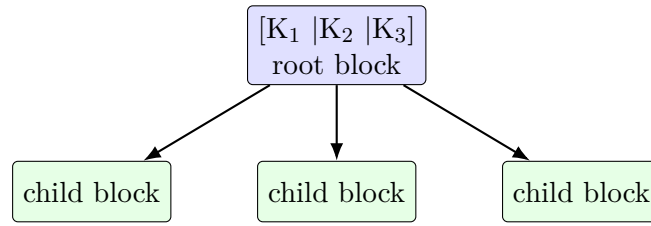
For a sorted sequence of n keys stored in blocks of size B , binary search performs $\Theta(\log_2 n)$ comparisons on the CPU. Its I/O cost is larger than its comparison count suggests: during the first iterations, the examined positions are separated by $n/2$, $n/4$, and so on. Until the search interval shrinks to at most B elements, each probe can address a different physical block. Thus,

$$\text{I/O}_{\text{binary search}} \approx \log_2 n - \log_2 B = \log_2 \left(\frac{n}{B} \right).$$

The approximation ignores alignment and the possibility that a block remains cached, but captures the dominant external-memory behavior.

2.3.2 B-Trees and B⁺-Trees

A B-Tree chooses the node size to match one physical block. Each node stores $\Theta(B)$ keys and child pointers, so its fan-out is $\Theta(B)$. A B⁺-Tree uses the same principle and normally stores records or record pointers in its leaves, making it especially suitable for range queries.



each node occupies approximately one physical block B

Figure 2.3: A block-oriented B-Tree with high fan-out.

Figure 2.3 shows the central layout decision: one I/O loads an entire node and supplies many routing keys and pointers. The height, and therefore the number of block reads for a point query, is approximately

$$h = \log_B \left(\frac{n}{B} \right) = \frac{\log_2(n/B)}{\log_2 B}.$$

More precisely, implementation details introduce ceilings and occupancy factors. The internal-node space is small compared with the leaves because

$$\text{internal nodes} \leq \frac{n}{B^2} + \frac{n}{B^3} + \cdots = \frac{n}{B(B-1)} \ll \frac{n}{B}.$$

2.3.3 Quantitative Comparison

Ignoring ceilings, the I/O speedup of a B-Tree over binary search is

$$\frac{\text{I/O}_{\text{binary search}}}{\text{I/O}_{\text{B-Tree}}} = \frac{\log_2(n/B)}{\log_2(n/B)/\log_2 B} = \log_2 B.$$

Table 2.3: Illustrative comparison for a large ordered dataset.

Parameter	Value
Dataset size	$n = 2^{40}$ elements, approximately 10^{12} records
Block size	$B = 2^{14} = 16\,384$ records
Binary-search I/O	$40 - 14 = 26$ block accesses
B-Tree I/O	$(40 - 14)/14 \approx 1.85$, hence about 2 block accesses

For millisecond-scale storage latency, the difference between approximately 26 and 2 block accesses changes a point query from hundreds of milliseconds to only a few milliseconds, subject to caching and hardware details.

2.4 I/O-Aware Algorithms and Virtual Paging

There are two complementary ways to manage transfers between fast and slow memory:

- **I/O-aware design (explicit):** The algorithm models B and the fast memory capacity M directly, and manages buffers, data layout, and transfers programmatically.
- **Virtual memory and paging (implicit):** The operating system and the hardware transparently move pages using page tables and replacement policies such as LRU approximations.

2.4.1 Paging with an Oversized Working Set

Assume that physical memory has capacity M and is divided into M/B frames. If the application requires

$$n = (1 + \varepsilon)M = M + \varepsilon M, \quad \varepsilon > 0,$$

then at least εM units of the working set must reside in secondary storage. Let α be the fraction of instructions that perform memory operations. A typical instruction profile is:

- fraction $1 - \alpha$: arithmetic or logical operations, with cost approximately one CPU cycle;
- fraction α : LOAD or STORE operations;

- probability $1 - P(\varepsilon)$: the referenced data are resident in RAM, with access time t_m ;
- probability $P(\varepsilon)$: the access causes a page fault, with service time t_{disk} .

The resulting average time per instruction is

$$T_{\text{avg}} = (1 - \alpha) \cdot 1 + \alpha [(1 - P(\varepsilon))t_m + P(\varepsilon)t_{\text{disk}}].$$

Since $t_m \ll t_{\text{disk}}$, the page-fault term can dominate:

$$T_{\text{avg}} \approx \alpha P(\varepsilon) t_{\text{disk}}.$$

This approximation is a warning about sensitivity, not an identity: when $P(\varepsilon)$ is zero, the baseline CPU and RAM terms still matter.

2.4.2 Numerical Thrashing Example

Take $\alpha = 0.3$, a page-fault penalty of $t_{\text{disk}} = 10^5$ clock cycles, and a fault probability of only $P(\varepsilon) = 10^{-3}$. Then the expected penalty is

$$T_{\text{penalty}} \approx 0.3 \cdot 10^{-3} \cdot 10^5 = 30 \text{ cycles per instruction.}$$

One page fault every thousand memory accesses can therefore add about 30 cycles to every instruction on average. Relative to a one-cycle baseline, this can produce an order-of-magnitude slowdown, potentially in the $10\times$ – $30\times$ range depending on the ordinary RAM latency and the actual working-set behavior. When the fault rate rises further, the process enters *thrashing*: the system spends most of its time servicing page faults instead of progressing with useful computation.

2.5 Key Takeaways

1. A modular stride visits all logical blocks only when the stride and the number of logical blocks are coprime.
2. Block-aware layouts and access patterns can change I/O complexity by a factor of B , even when CPU work is unchanged.
3. B-Trees reduce external-memory search depth by matching their fan-out to the physical block size.

4. Hardware acceleration improves linear and polynomial algorithms in different ways, but gives only an additive improvement to exponential algorithms.
5. Virtual memory is convenient, but an oversized working set can turn a small page-fault probability into a dominant performance cost.

Chapter 3

Permutation, Sorting, and Multiway Merge Sort in External Memory

Lecture 3

Permutation vs. Sorting, Multiway Merge Sort, Snow Plow, and Disk Striping

Date: September 18, 2026

In this chapter, we examine the fundamental relationship between the permutation problem and the sorting problem in the two-level memory hierarchy, contrasting the limitations of the RAM model with block-based storage systems. We analyze the external Binary Merge Sort, highlighting its memory underutilization bottleneck, and develop in detail the *Multiway Merge Sort* (k -Way Merge Sort) that achieves the asymptotic lower bound of Aggarwal and Vitter. Finally, we explore two core Algorithm Engineering techniques for secondary storage: optimizing initial runs via the *Snow Plow* algorithm (*Replacement Selection*) and scaling parallel throughput across multiple disks via *Disk Striping*.

3.1 The Permutation vs. Sorting Problem

3.1.1 Problem Formulation

We are given two vectors of size n :

- A data array $S[1 \dots n]$.

- A permutation vector $\pi[1 \dots n]$, containing a bijective permutation of the indices $\{1, 2, \dots, n\}$.

The objective is to rearrange the elements of S into a new array S' such that each original element $S[i]$ is placed into the destination position specified by $\pi[i]$:

$$S'[\pi[i]] = S[i], \quad \forall i \in \{1, \dots, n\}. \quad (3.1)$$

Example 3.1.1 (Index Permutation). Consider an array of 4 elements and its corresponding permutation vector:

$$\begin{array}{l} \text{Indices: } 1 \quad 2 \quad 3 \quad 4 \\ S = [A, \quad B, \quad C, \quad D] \\ \pi = [3, \quad 1, \quad 2, \quad 4] \end{array}$$

Applying the definition element by element:

$$\begin{aligned} S'[\pi[1]] &= S'[3] = A, \\ S'[\pi[2]] &= S'[1] = B, \\ S'[\pi[3]] &= S'[2] = C, \\ S'[\pi[4]] &= S'[4] = D. \end{aligned}$$

The resulting permuted array is therefore:

$$S' = [B, \quad C, \quad A, \quad D].$$

In [Figure 3.1](#), the mapping from the source array to the destination array is illustrated visually.

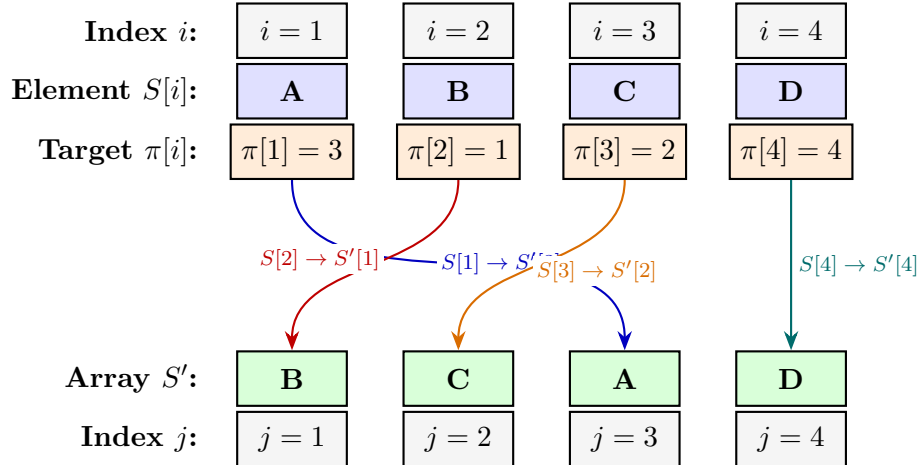


Figure 3.1: Permutation mapping: each element $S[i]$ is relocated to the destination index specified by $\pi[i]$ in array S' .

3.1.2 Naive Implementation on the RAM Model

In the standard RAM model, where any read or write access to arbitrary memory locations costs $\mathcal{O}(1)$ time, the permutation is implemented in a single linear pass:

Algorithm 1: Direct Permutation on RAM

Input: Array $S[1 \dots n]$, permutation vector $\pi[1 \dots n]$

Output: Permuted array S in place

```

1 for  $i \leftarrow 1$  to  $n$  do
2    $S'[\pi[i]] \leftarrow S[i]$ 
3  $S \leftarrow \text{copy}(S')$ 

```

The time complexity in the RAM model is strictly linear:

$$T_{\text{RAM}}(n) = \Theta(n).$$

Therefore, in internal memory, permuting is asymptotically faster than sorting by comparisons, which requires $\Omega(n \log n)$ operations.

3.1.3 Simulation Principle (RAM to Two-Level Memory)

Theorem 3.1.1 (Simulation Theorem). *Any sequential algorithm formulated for the RAM model with time complexity $T(n)$ incurs at most $\mathcal{O}(T(n))$ block transfers (I/Os) in the two-level memory model with parameters (M, B) .*

Proof Sketch. In the worst case, every single RAM operation accessing memory addresses a memory cell that resides in a block not currently resident in internal cache M . Each such access triggers a block transfer from disk, resulting in exactly 1 I/O operation. Multiplying the total step count $T(n)$ by the upper bound of 1 I/O per step yields the trivial bound of $\mathcal{O}(T(n))$ I/Os. $\square$

Although the simulation principle guarantees that any RAM algorithm can be ported to external memory, the bound $\mathcal{O}(T(n))$ is often thoroughly sub-optimal because it fails to exploit the block aggregation factor B .

3.1.4 The Relationship Between Sorting and Permutation

In the two-level memory hierarchy, the complexity relationship changes fundamentally, as summarized in [Table 3.1](#):

Problem	RAM Model	Two-Level Memory (I/O)
Permutation Perm(n)	$\Theta(n)$	$\Theta\left(\min\left\{n, \text{Sort}(n, M, B)\right\}\right)$
Sorting Sort(n)	$\Theta(n \log_2 n)$	$\Theta\left(\frac{n}{B} \log_{M/B} \frac{n}{B}\right)$

Table 3.1: Asymptotic complexity comparison between RAM and External Memory models.

To understand this equivalence in block-based storage, consider the two reductions:

1. **Sorting via Permutation:** Given an array S , sorting requires computing the permutation π_S that puts keys in order, and then applying π_S to S .
2. **Permuting via Sorting (Pair Reduction Algorithm):** Permuting an array S can be converted into an external sorting problem via the following three-step procedure:
 - **Step 1 (Scan and Pair Creation):** Perform parallel linear scans over S and π to construct a set of n records containing $\langle S[i], \pi[i] \rangle$. Since this requires only streaming sequential I/O, the cost is:

$$\text{I/O}_{\text{Step 1}} = \mathcal{O}\left(\frac{n}{B}\right).$$

- **Step 2 (Sorting by Destination):** Sort the n records using the second component $\pi[i]$ as the sorting key. The cost is identical to external sorting:

$$\text{I/O}_{\text{Step 2}} = \text{Sort}(n, M, B).$$

- **Step 3 (Scan and Extraction):** Since the records are now sorted by increasing target index, a simple sequential scan extracts the first coordinate $S[i]$ into the destination array S' . The cost is:

$$\text{I/O}_{\text{Step 3}} = \mathcal{O}\left(\frac{n}{B}\right).$$

The overall I/O cost of this reduction is:

$$\text{I/O}_{\text{Sort-based}} = \mathcal{O}\left(\frac{n}{B}\right) + \text{Sort}(n, M, B) + \mathcal{O}\left(\frac{n}{B}\right) = \Theta(\text{Sort}(n, M, B)). \quad (3.2)$$

Example 3.1.2 (Step-by-Step Pair Reduction Trace on Concrete Arrays). To visualize the reduction with concrete values, consider the instance from [Example 3.1.1](#):

$$S = [A, B, C, D], \quad \pi = [3, 1, 2, 4].$$

The algorithm executes across three sequential states of the record array:

- **Step 1 (Pairing):** Sequentially pair each item with its designated target index:

$$P = [\langle A, 3 \rangle, \langle B, 1 \rangle, \langle C, 2 \rangle, \langle D, 4 \rangle].$$

- **Step 2 (External Sorting on $\pi[i]$):** Sort the record array using the second coordinate (target destination) as the sorting key:

$$P_{\text{sorted}} = [\langle B, 1 \rangle, \langle C, 2 \rangle, \langle A, 3 \rangle, \langle D, 4 \rangle].$$

- **Step 3 (Sequential Extraction):** Stream through the sorted records and project their first coordinates to form the destination array:

$$S' = [B, C, A, D],$$

which correctly fulfills the required permutation since $S'[1] = B$, $S'[2] = C$, $S'[3] = A$, $S'[4] = D$.

The pair reduction pipeline is illustrated in [Figure 3.2](#).

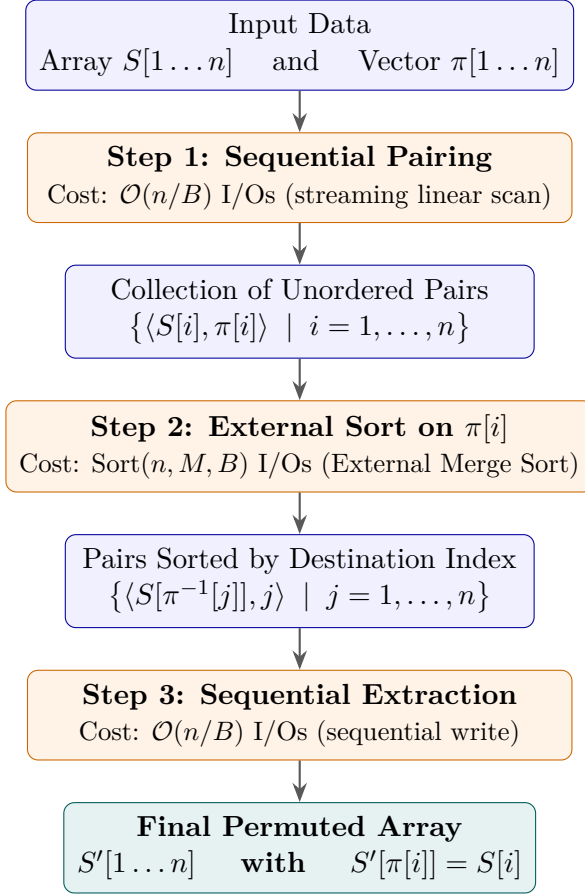


Figure 3.2: Operational workflow of the pair-reduction permutation algorithm.

3.2 Permutation Complexity in External Memory

The fundamental lower bound for permutation in the two-level memory model was established by Aggarwal and Vitter [2]:

$$\text{Perm}(n, M, B) = \Theta \left(\min \left\{ n, \frac{n}{B} \log_{M/B} \left(\frac{n}{B} \right) \right\} \right). \quad (3.3)$$

Letting $L = \log_{M/B}(n/B)$ denote the number of merge passes required by multiway merge sort:

$$\text{Sort}(n, M, B) = \Theta \left(\frac{n}{B} \cdot L \right). \quad (3.4)$$

3.2.1 Corner Case Analysis

Comparing the direct naive algorithm with the sorting-based reduction yields two distinct regimes determined by the relationship between block size B and pass count L :

- **Case 1** ($n \geq \frac{n}{B} \cdot L \iff B \geq L$): The block size B is greater than or equal to the depth of the merge tree. In this setting, sorting the pairs requires fewer I/Os than random single-element access:

$$\text{Perm}(n, M, B) = \Theta(\text{Sort}(n, M, B)).$$

- **Case 2** ($n < \frac{n}{B} \cdot L \iff B < L$): If the block size were pathologically small compared to the overall dataset ($B < L$), direct element-by-element random writing (at most n I/Os) would be asymptotically cheaper than sorting:

$$\text{Perm}(n, M, B) = \Theta(n).$$

3.2.2 Numerical Verification on Realistic Architectural Parameters

To establish which regime governs real-world computing systems, consider extreme yet realistic parameters:

- Fast internal memory: $M = 16 \text{ GB} = 2^4 \cdot 2^{30} = 2^{34}$ elements.
- Disk block size: $B = 32 \text{ KB} = 2^5 \cdot 2^{10} = 2^{15}$ elements.
- Input dataset size: $n = 2^{60}$ elements (Exabyte scale).

We calculate the merge depth L :

$$L = \log_{M/B} \left(\frac{n}{M} \right) = \frac{\log_2(n/M)}{\log_2(M/B)}.$$

Substituting the exponential values:

$$\begin{aligned} \frac{n}{M} &= \frac{2^{60}}{2^{34}} = 2^{26} \implies \log_2 \left(\frac{n}{M} \right) = 26, \\ \frac{M}{B} &= \frac{2^{34}}{2^{15}} = 2^{19} \implies \log_2 \left(\frac{M}{B} \right) = 19. \end{aligned}$$

Hence:

$$L = \frac{26}{19} \approx 1.36 < 2.$$

Because $B = 2^{15} = 32\,768$ and $L \leq 2$, the condition $B \geq L$ holds by more than four orders of magnitude ($32\,768 \gg 2$). Consequently, in all practical systems, permutation is strictly bound by sorting complexity:

$$\text{Perm}(n) = \Theta(\text{Sort}(n)).$$

3.3 External Memory Binary Merge Sort

The classical recursive Merge Sort algorithm extends naturally to external memory by recursively dividing arrays, ordering runs, and merging sorted sub-arrays:

Algorithm 2: Binary Merge Sort

Input: Array S , boundaries i, j

Output: Sorted sub-array $S[i \dots j]$

```

1 if  $i < j$  then
2    $m \leftarrow \lfloor (i + j) / 2 \rfloor$ 
3    $\text{MS}(S, i, m)$ 
4    $\text{MS}(S, m + 1, j)$ 
5    $\text{MERGE}(S, i, m, j)$ 

```

3.3.1 Buffer Architecture in the Merge Procedure

Given two consecutive sorted runs of length l , the merge procedure produces a single consolidated run of length $2l$ into an auxiliary array S' .

- **In the RAM Model:** The operation performs at most $2l$ comparisons and $2l$ memory writes, costing $\Theta(l)$ CPU steps.
- **In the Two-Level Model:** To minimize block transfers, internal memory M is partitioned into 3 dedicated buffers, each having size exactly equal to one block B :
 1. **Input Buffer 1:** holds one block (size B) of the first sorted run.
 2. **Input Buffer 2:** holds one block (size B) of the second sorted run.
 3. **Output Buffer:** accumulates merged elements in sorted order until full (B elements).

Figure 3.3 illustrates the internal memory allocation and streaming I/O pattern during the binary merge.

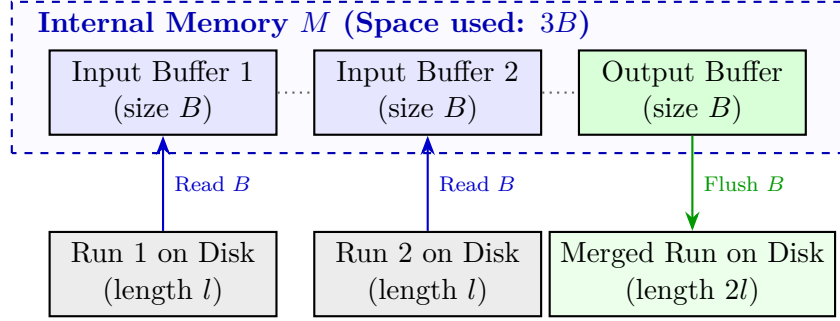


Figure 3.3: Internal memory buffer organization during binary external merge.

The execution protocol proceeds as follows:

1. The leading elements of the two input buffers are compared in CPU; the smaller key is appended to the output buffer.
2. Whenever an input buffer is exhausted, a disk read loads the next block of that run ($\frac{l}{B}$ I/Os per input run).
3. As soon as the output buffer fills up with B items, it is flushed to disk in a single write operation ($\frac{2l}{B}$ write I/Os overall).

The total I/O cost of merging two runs of length l is:

$$\text{I/O}_{\text{Merge}}(l) = \frac{l}{B} + \frac{l}{B} + \frac{2l}{B} = \frac{4l}{B} = \Theta\left(\frac{l}{B}\right). \quad (3.5)$$

3.3.2 Recursion Tree Complexity

Recursion terminates once sub-array sizes fit entirely within internal memory M . At this base level, each chunk of size M is read into memory, sorted using an in-memory algorithm (e.g., QuickSort or HeapSort) with zero additional I/O, and written back to disk using $\mathcal{O}(M/B)$ I/Os.

This yields:

$$N_{\text{run}} = \frac{n}{M} \text{ initial sorted runs of size } M.$$

The binary merge tree has height:

$$h = \log_2\left(\frac{n}{M}\right) \text{ levels.} \quad (3.6)$$

Figure 3.4 depicts the recursion structure of binary external merge sort.

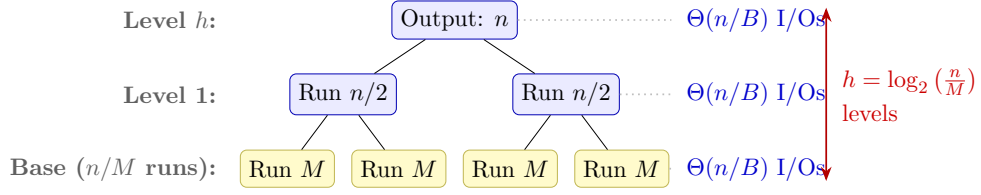


Figure 3.4: Recursion tree of external memory Binary Merge Sort with level I/O costs.

At each level of the tree, all n elements are streamed exactly once, requiring $\Theta(n/B)$ I/Os per level. Multiplying by tree height gives the overall complexity:

$$\text{I/O}_{\text{BinaryMergeSort}} = \mathcal{O}\left(\frac{n}{B} \log_2\left(\frac{n}{M}\right)\right). \quad (3.7)$$

3.3.3 Sub-optimality and Bottlenecks

Although significantly more efficient than naive approaches, this design suffers from a major structural bottleneck:

- **Drastic internal memory underutilization:** During merging, only 3 blocks ($3B$ elements) are occupied in fast memory M . The remaining capacity ($M - 3B$) sits completely idle.
- **Fixed logarithm base:** The base of the logarithm is fixed at 2, whereas memory M could accommodate up to:

$$k \approx \frac{M}{B} - 1 \text{ concurrent input streams.}$$

- **Loss of the $\log_2(M/B)$ convergence factor:** By failing to perform k -way merging (*Multiway Merge Sort*), the algorithm executes substantially more passes, falling short of the optimal lower bound:

$$\mathcal{O}\left(\frac{n}{B} \log_{M/B}\left(\frac{n}{B}\right)\right).$$

Figure 3.5 visually contrasts this internal memory underutilization against full buffer saturation.

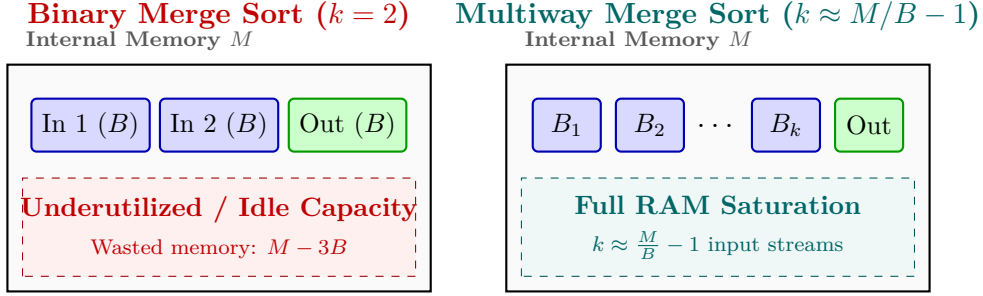


Figure 3.5: Internal memory allocation efficiency: severe underutilization in Binary Merge Sort versus full buffer saturation in Multiway Merge Sort.

Overcoming this structural limitation leads naturally to the multiway merging architecture, which we investigate in detail in the following sections.

3.4 Asymptotic Equivalence of the External Sorting Cost

In the computational analysis of secondary memory algorithms, the I/O complexity of external sorting is expressed in the literature [2] using two seemingly distinct formulations:

$$\Theta\left(\frac{n}{B} \log_{M/B}\left(\frac{n}{M}\right)\right) \quad \text{or} \quad \mathcal{O}\left(\frac{n}{B} \log_{M/B}\left(\frac{n}{B}\right)\right). \quad (3.8)$$

These two expressions are asymptotically equivalent for any problem instance where $n \geq M$.

To establish this equivalence formally, we apply the logarithmic product rule to expand the logarithmic term $\log_{M/B}(n/B)$:

$$\begin{aligned} \log_{M/B}\left(\frac{n}{B}\right) &= \log_{M/B}\left(\frac{n}{M} \cdot \frac{M}{B}\right) \\ &= \log_{M/B}\left(\frac{n}{M}\right) + \log_{M/B}\left(\frac{M}{B}\right) \\ &= \log_{M/B}\left(\frac{n}{M}\right) + 1. \end{aligned} \quad (3.9)$$

Multiplying the entire equation by the common scale factor $\frac{n}{B}$, we obtain:

$$\frac{n}{B} \log_{M/B}\left(\frac{n}{B}\right) = \frac{n}{B} \log_{M/B}\left(\frac{n}{M}\right) + \frac{n}{B}. \quad (3.10)$$

Algorithmically, the additional linear scan term $\frac{n}{B}$ corresponds precisely to the initial pass (Phase 0) that creates the initial sorted runs by loading chunks of size M into RAM, sorting them internally, and flushing them back to disk.

Whenever $n \gg M$, we have $\log_{M/B}(n/M) \geq 1$, which implies that the additive $+1$ term is readily subsumed by the hidden constant within asymptotic Θ -notation:

$$\Theta\left(\frac{n}{B} \log_{M/B}\left(\frac{n}{B}\right)\right) = \Theta\left(\frac{n}{B} \left(\log_{M/B}\left(\frac{n}{M}\right) + 1\right)\right) = \Theta\left(\frac{n}{B} \log_{M/B}\left(\frac{n}{M}\right)\right). \quad (3.11)$$

Consequently, one may freely choose either $\frac{n}{M}$ or $\frac{n}{B}$ as the argument of the logarithm without altering the asymptotic validity of the result.

3.5 Multiway Merge Sort (k -Way Merge Sort)

In the external *Binary Merge Sort* presented in [section 3.3](#), only 3 blocks of internal memory were concurrently active, wasting the remaining $M - 3B$ elements of fast storage. The **Multiway Merge Sort** (or k -way merge sort) overcomes this severe bottleneck by utilizing the entire capacity M to merge k sorted runs simultaneously in a single pass (*fan-in* equal to k).

3.5.1 Internal Memory Configuration and Layout

Primary memory M is partitioned into blocks of transfer size B :

- $k = \frac{M}{B} - 1$ **input buffers**, each having capacity B , statically associated with one active run on external disk storage.
- 1 **output buffer**, also of capacity B , dedicated to incrementally accumulating sorted elements before performing an I/O flush to disk.
- An auxiliary in-memory data structure: a **Min-Heap** (or priority queue) of cardinality k . Each heap node stores the current leading key of one of the k active input buffers along with an identifier indicating its source buffer.

[Figure 3.6](#) illustrates the architectural buffer layout and the data flow coordinated by the Min-Heap.

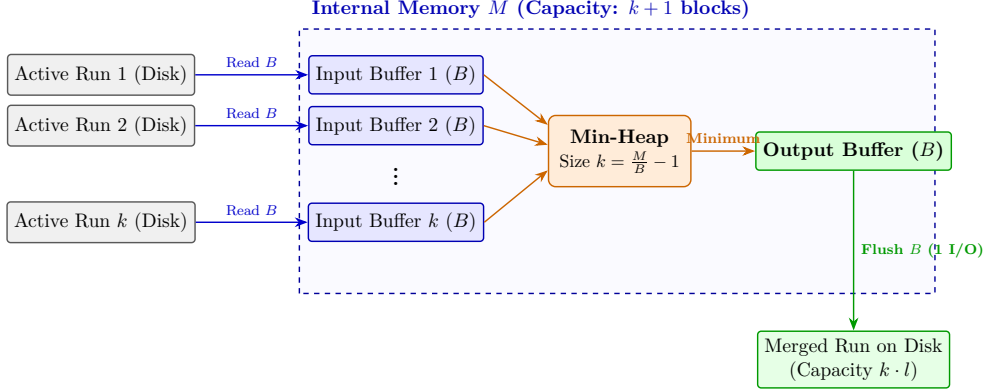


Figure 3.6: Internal memory layout during multiway merging (k -Way Merge Sort) with coordinating Min-Heap.

The Min-Heap allows extracting the global minimum among all k streams in $\mathcal{O}(\log_2 k)$ internal CPU comparisons. When the input buffer supplying the minimum is exhausted, a single disk read I/O is performed to load the next block of B elements from that run. Similarly, each time the output buffer accumulates B sorted elements, a single physical write I/O flushes the block to secondary storage.

3.5.2 Algorithm Execution and Tree Height Evolution

The execution proceeds across two distinct phases:

1. **Phase 0 (Initial Run Formation):** The unsorted array of n elements stored on disk is streamed in chunks of size M . Each chunk of M elements is brought into RAM, sorted using an optimal in-memory algorithm (e.g., QuickSort or HeapSort), and flushed back to disk:

- Number of initial sorted runs:

$$N_0 = \frac{n}{M}. \quad (3.12)$$

- I/O cost of Phase 0: full read and write scan across all n elements:

$$\text{I/O}_{\text{RunFormation}} = 2 \cdot \frac{n}{B} = \mathcal{O}\left(\frac{n}{B}\right). \quad (3.13)$$

2. **Subsequent Multiway Merging Passes (k -Way Merge):** At each level of the recursion tree, existing runs are partitioned into groups of size k . Each group of k runs, each of current length l , is merged simultaneously to yield a single consolidated run of size $k \cdot l$:

- I/O cost for merging a single group of runs: $\mathcal{O}\left(\frac{k \cdot l}{B}\right)$.
- Total I/O cost per merge level: because every element belongs to exactly one group per level, all n elements are read and written exactly once per pass, resulting in:

$$\text{I/O}_{\text{Level}} = 2 \cdot \frac{n}{B} = \mathcal{O}\left(\frac{n}{B}\right). \quad (3.14)$$

At the i -th level of the multiway merge tree, every run has size $k^i \cdot M$. The merge process terminates when only a single consolidated run of size n remains:

$$k^i \cdot M = n \implies k^i = \frac{n}{M} \implies i = \log_k\left(\frac{n}{M}\right) \text{ levels.} \quad (3.15)$$

Setting the fan-in to its maximum feasible capacity $k \approx \frac{M}{B} - 1 = \Theta\left(\frac{M}{B}\right)$, we obtain:

$$\text{Number of Merge Passes} = \mathcal{O}\left(\log_{M/B}\left(\frac{n}{M}\right)\right). \quad (3.16)$$

In Figure 3.7, we illustrate the hierarchy of k -way merge levels, highlighting the expanded fan-in and per-level I/O scan costs throughout the recursion tree.

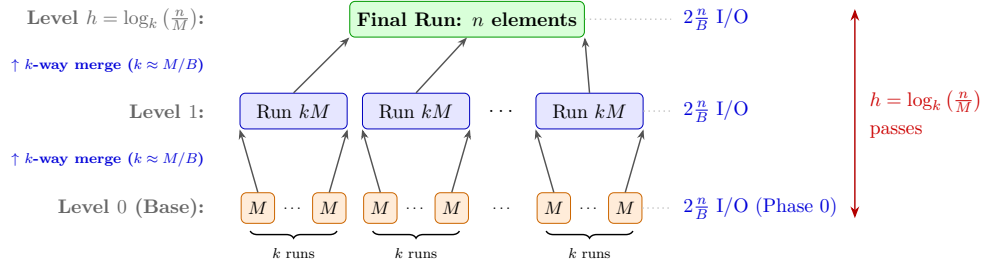


Figure 3.7: Level evolution and k -way merge tree in Multiway Merge Sort.

Combining both phases gives the overall I/O complexity:

$$\begin{aligned} \text{I/O}_{\text{Total}} &= \underbrace{\mathcal{O}\left(\frac{n}{B}\right)}_{\text{Run Formation}} + \underbrace{\mathcal{O}\left(\frac{n}{B} \log_{M/B}\left(\frac{n}{M}\right)\right)}_{\text{Merging Passes}} \\ &= \mathcal{O}\left(\frac{n}{B} \log_{M/B}\left(\frac{n}{B}\right)\right) \text{ I/O operations.} \end{aligned} \quad (3.17)$$

This matches the asymptotic lower bound for hierarchical memory sorting established by Aggarwal and Vitter [2], proving that Multiway Merge Sort is asymptotically optimal.

3.6 Optimizing Run Formation: The Snow Plow (*Replacement Selection*)

While standard Multiway Merge Sort creates initial runs of length bounded by memory capacity M , the tree depth $\lceil \log_{M/B}(n/M) \rceil$ depends directly on the initial number of runs. Producing runs substantially longer than M flattens the merge tree and can eliminate an entire I/O pass across petabyte-scale datasets.

A foundational technique designed to achieve this is **Replacement Selection**, historically known by the vivid metaphor of the **Snow Plow** [1].

3.6.1 Operational Mechanism of Replacement Selection

Rather than statically filling memory M , sorting it, and flushing it in bulk, the algorithm keeps fast memory M fully occupied at all times by dynamically splitting it into two logical partitions:

- **Active Elements (Current Min-Heap):** elements whose keys are greater than or equal to the last element emitted to the current run on disk. These elements are valid candidates to append to the active run.
- **Frozen Elements (U):** newly ingested elements whose keys are strictly smaller than the last element emitted. Appending them would violate the sorted invariant; hence they are “frozen” in RAM, waiting for the subsequent run.

Figure 3.8 shows the dynamic memory partitioning during the snow plow sweep.

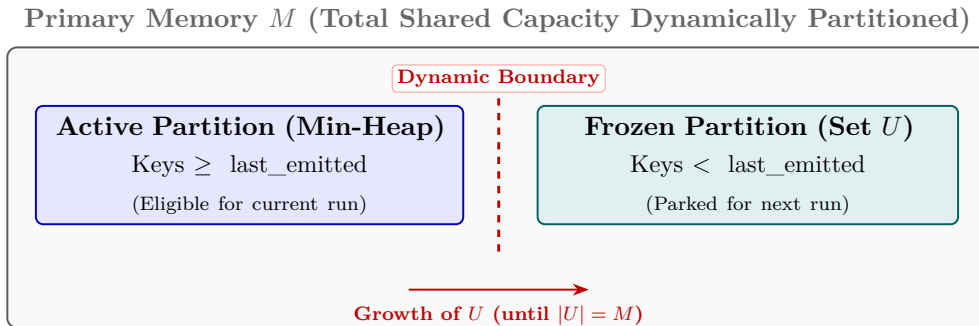


Figure 3.8: Dynamic partitioning of memory M during Replacement Selection (Snow Plow).

The algorithm executes the following element-wise loop:

1. Extract the minimum active element from the root of the Min-Heap, write it to the output buffer, and record its key as `last_emitted`.
2. Ingest the next item x from the unsorted input stream:
 - If $x \geq \text{last_emitted}$, insert x into the active Min-Heap.
 - If $x < \text{last_emitted}$, assign x to the frozen set U .
3. The active run terminates when the active heap becomes empty, i.e., when all M memory slots are occupied by frozen elements ($|U| = M$). At this moment, set U is transformed into the new active Min-Heap, a new run file is opened on disk, and the cycle repeats.

3.6.2 Step-by-Step Execution Trace

Consider internal memory capacity $M = 3$ and an input stream:

$$S = [1, 7, 5, 3, 2, 9, 0, 4, \dots].$$

Table 3.2 traces the step-by-step state of memory and the output stream.

Table 3.2: Execution trace of Replacement Selection with capacity $M = 3$.

Step	Active	Frozen (U)	Action Performed	Emitted	Run
0	{1, 5, 7}	$\emptyset$	Initial load of $M = 3$ items	—	[]
1	{5, 7}	$\emptyset$	Emit 1; read 3. Since $3 \geq 1$, remains active	1	[1]
2	{5, 7}	$\emptyset$	Emit 3; read 2. Since $2 < 3$, 2 goes to U	3	[1, 3]
3	{7, 9}	{2}	Emit 5; read 9. Since $9 \geq 5$, remains active	5	[1, 3, 5]
4	{9}	{2, 0}	Emit 7; read 0. Since $0 < 7$, 0 goes to U	7	[1, 3, 5, 7]
5	$\emptyset$	{2, 0, 4}	Emit 9; read 4. Since $4 < 9$, 4 goes to U	9	[1, 3, 5, 7, 9]

At Step 5, active memory is completely depleted and frozen items fill all M slots: $|U| = M = 3$. The current run closes with a length of **5 items**—significantly exceeding physical memory capacity $M = 3$. The frozen set $\{0, 2, 4\}$ immediately forms the seed heap for the subsequent run.

Note on Production Systems. In production databases and file systems, transfers are performed using blocks of size B rather than single elements, utilizing prefetching buffers and write accumulation buffers (*double buffering*) without altering the asymptotic I/O behavior.

3.6.3 Average Run Length Theorem

Theorem 3.6.1 (Average Run Length in Snow Plow). *Assuming that input keys are independent and identically distributed random variables drawn from a continuous uniform distribution, the average run length produced by Replacement Selection is asymptotically $2M$.*

Heuristic Proof. We analyze the steady-state stochastic process:

1. At the start of a run, memory holds M elements.
2. Let τ denote the number of additional items read from the input before the run ends. The total run length is $M + \tau$.
3. The run concludes when the frozen buffer reaches full capacity: $|U| = M$.
4. By distributional symmetry and statistical independence, each incoming element has probability $P = \frac{1}{2}$ of being smaller than the last emitted element (landing in U) and probability $P = \frac{1}{2}$ of being greater than or equal to it (joining the active run).
5. The expected number of items placed in U after reading τ elements is:

$$\mathbb{E}[|U|] = \tau \cdot P(\text{placed into } U) = \tau \cdot \frac{1}{2}. \quad (3.18)$$

6. Equating this expectation to the termination threshold $\mathbb{E}[|U|] = M$:

$$\frac{\tau}{2} = M \implies \tau = 2M. \quad (3.19)$$

Consequently, the expected average run length is approximately $2M$. $\square$

Doubling the average run length ($\approx 2M$) cuts the initial number of runs in half $\left(\frac{n}{2M}\right)$, directly decreasing the required number of full I/O passes across the dataset.

3.7 Parallelism Across Multiple Disks: *Disk Striping*

In modern data centers and high-performance computing clusters, secondary storage is distributed across D independent physical disk drives or SSD units.

3.7.1 Parallel External Sorting Lower Bound

When a storage architecture incorporates D parallel disk units, each capable of transferring one block B per I/O cycle, the Aggarwal-Vitter [2] lower bound scales with an ideal factor D in the denominator:

$$\text{Sort}_D(n, M, B) = \Omega\left(\frac{n}{D \cdot B} \log_{M/B}\left(\frac{n}{M}\right)\right) \text{ parallel I/O steps.} \quad (3.20)$$

However, designing an algorithm that achieves this lower bound using asynchronous independent disks is remarkably intricate: one must continuously prevent disk contention and bottlenecking where access concentrates on a single drive while $D - 1$ drives sit idle.

3.7.2 The Disk Striping Technique

To exploit physical parallelism without introducing complex asynchronous disk schedulers, systems employ **Disk Striping** (analogous to RAID 0 architecture):

- The D physical disks $d_1, d_2, \dots, d_D$ are logically aggregated into a single unified volume.
- A **logical super-block** of aggregate size:

$$B' = D \cdot B \quad (3.21)$$

is defined by combining one physical block of size B stored at the identical track or cylinder position on each of the D drives.

- Every I/O transfer operates on an entire super-block of size B' , distributing the I/O load synchronously and uniformly across all D channels in a single parallel transfer cycle.

Figure 3.9 illustrates the mapping of super-block B' onto the D underlying physical disks.

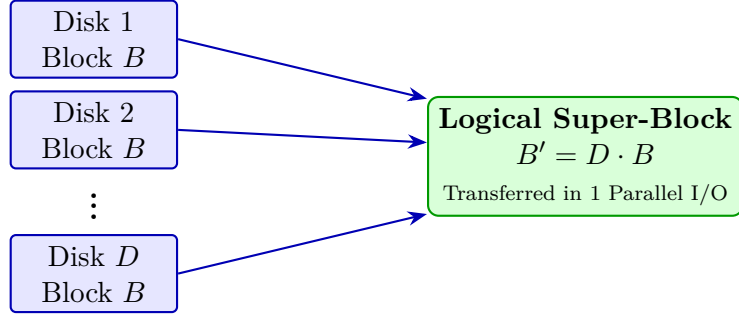


Figure 3.9: Disk striping layout: aggregating D physical blocks into a single logical super-block B' .

Executing Multiway Merge Sort using logical block size B' , the number of super-blocks is $\frac{n}{B'} = \frac{n}{D \cdot B}$, while the memory fan-in capacity becomes $\frac{M}{B'} = \frac{M}{D \cdot B}$. The parallel I/O complexity is:

$$\text{I/O}_{\text{Striping}} = \mathcal{O}\left(\frac{n}{B'} \log_{M/B'}\left(\frac{n}{M}\right)\right) = \mathcal{O}\left(\frac{n}{D \cdot B} \log_{\frac{M}{D \cdot B}}\left(\frac{n}{M}\right)\right). \quad (3.22)$$

3.7.3 Slowdown Analysis

Although Disk Striping achieves full bandwidth utilization across all D channels, it reduces the available merging fan-in from $\frac{M}{B}$ down to $\frac{M}{D \cdot B}$, because each open run requires a buffer of size $B' = D \cdot B$.

To quantify this loss of efficiency relative to the optimal independent-disk lower bound, we compute the ratio of their I/O complexities (*Slowdown*):

$$\text{Slowdown} = \frac{\text{I/O}_{\text{Striping}}}{\text{I/O}_{\text{Ideal}}} = \frac{\frac{n}{D \cdot B} \log_{\frac{M}{D \cdot B}}\left(\frac{n}{M}\right)}{\frac{n}{D \cdot B} \log_{\frac{M}{B}}\left(\frac{n}{M}\right)} = \frac{\log_{\frac{M}{D \cdot B}}\left(\frac{n}{M}\right)}{\log_{\frac{M}{B}}\left(\frac{n}{M}\right)}. \quad (3.23)$$

Changing the logarithm base to 2 using $\log_a x = \frac{\log_2 x}{\log_2 a}$:

$$\text{Slowdown} = \frac{\frac{\log_2(n/M)}{\log_2(M/(D \cdot B))}}{\frac{\log_2(n/M)}{\log_2(M/B)}} = \frac{\log_2(M/B)}{\log_2(M/(D \cdot B))} = \frac{\log_2(M/B)}{\log_2(M/B) - \log_2 D}. \quad (3.24)$$

Dividing numerator and denominator by $\log_2(M/B)$:

$$\text{Slowdown} = \frac{1}{1 - \frac{\log_2 D}{\log_2(M/B)}}. \quad (3.25)$$

3.7.4 Asymptotic Regimes and Physical Limitations

The behavior of the slowdown factor is governed entirely by the dimensionless ratio:

$$\rho = \frac{\log_2 D}{\log_2(M/B)}. \quad (3.26)$$

Two limiting operating regimes arise:

1. **Ample Memory Regime** ($M \rightarrow \infty$ or $D \ll \frac{M}{B}$): When RAM is substantially larger than the number of attached disk drives, the memory fan-in factor dominates:

$$\rho = \frac{\log_2 D}{\log_2(M/B)} \rightarrow 0 \implies \text{Slowdown} \rightarrow \frac{1}{1-0} = 1. \quad (3.27)$$

In this regime, Disk Striping suffers virtually no degradation and attains the asymptotic performance of the theoretical independent-disk system, representing an optimal engineering tradeoff due to its implementation simplicity.

2. **Memory Saturation Regime** ($\log_2 D \rightarrow \log_2(M/B)$): When the drive count D approaches the maximum number of blocks that fit into internal memory ($D \approx \frac{M}{B}$), the ratio ρ approaches 1. The denominator of (3.25) tends to zero, causing the expression to diverge:

$$\lim_{\rho \rightarrow 1^-} \frac{1}{1-\rho} = +\infty. \quad (3.28)$$

Physical interpretation: If $D \geq \frac{M}{B}$, internal memory cannot accommodate even a single super-block $B' = D \cdot B$ per active input stream, rendering striping physically impossible and causing the multiway merge to collapse.

Chapter 4

Lower Bounds for Sorting, Dual-Pivot Quicksort, and Bounded Quicksort

Lecture 4

*Lower Bounds (RAM vs Two-Level Model), Dual-Pivot Quicksort, and
Bounded Quicksort*

Date: September 25, 2026

This chapter investigates the theoretical and engineering analysis of complexity lower bounds for the fundamental problem of comparison-based sorting, contrasting the classical RAM model with the two-level memory model of Aggarwal and Vitter. Subsequently, the focus shifts to modern architectural developments of the *Quicksort* algorithm: we analyze the duplicate keys problem and the *Dual-Pivot Quicksort* algorithm introduced by Yaroslavskiy, followed by a formal treatment of the *Bounded Quicksort* (BQS) variant. We rigorously prove how tail recursion elimination and bounded partitioning guarantee an auxiliary stack space strictly bounded by $\Theta(\log_2 n)$, while achieving optimal cache efficiency via an *Insertion Sort* cutoff.

4.1 RAM Model: Lower Bounds and Decision Trees

In the standard RAM (*Random Access Machine*) model of computation, a comparison-based algorithm cannot inspect the underlying bitwise representation of keys to extract digits or radix components (as done in *Radix Sort*

algorithms). Instead, it may only acquire information regarding the relative ordering of elements via binary comparisons of the form:

$$A < B, \quad A = B, \quad \text{or} \quad A > B. \quad (4.1)$$

Assuming, without loss of generality for lower bound derivations, that all input keys are pairwise distinct, each elementary comparison yields a strictly dichotomous outcome ($<$ or $>$).

4.1.1 The Decision Tree Model

The family of deterministic comparison-based algorithms is formally modeled through the abstract mathematical structure of a **Decision Tree**:

- **Internal nodes:** Each internal node models a single elementary comparison between a pair of input elements or positions, conventionally denoted by $[A : B]$. Since all keys are distinct, exactly two directed edges leave the node, corresponding to the two mutually exclusive outcomes: the left edge associated with $A < B$ and the right edge with $A > B$. Consequently, the decision tree is a proper binary tree.
- **Root-to-leaf path:** For any given input instance of size n , the deterministic algorithm traces a unique directed path from the root down to a leaf. Each node visited along this trajectory corresponds to a comparison executed by the algorithm; the length of the path (the number of traversed edges) reflects the exact comparison count for that specific configuration. The length of the longest root-to-leaf path corresponds to the **height** t of the decision tree and quantifies the worst-case running time $T(n)$.
- **Leaves:** Each leaf corresponds to algorithmic termination and encapsulates the final solution or output configuration of the computational problem (e.g., the permutation of indices that sorts the array, or the index at which the target key is located).

Figure 4.1 illustrates the general topological structure of a binary decision tree for a generic comparison-based problem.

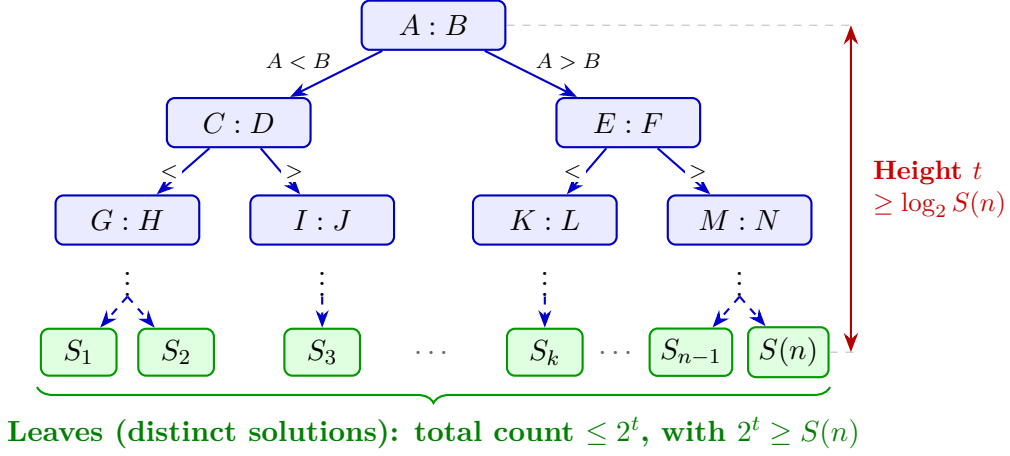


Figure 4.1: Binary decision tree model for comparison-based algorithms. Internal nodes represent elementary pairwise comparisons; leaves represent admissible distinct solutions $(S_1, S_2, \dots, S(n))$. The longest path determines the worst-case running time.

4.1.2 The General Information-Theoretic Lower Bound

The fundamental relationship between the number of distinct solutions of a problem and the minimum number of comparisons necessary to solve it is established by the following information-theoretic theorem.

Theorem 4.1.1 (General Information-Theoretic Lower Bound). *Let P be a computational problem solvable via a deterministic comparison-based algorithm. If for an input of size n the problem admits $S(n)$ distinct possible solutions (each of which must be returned by the algorithm for suitable input configurations), then any deterministic comparison-based algorithm requires in the worst case:*

$$T(n) = \Omega(\log_2 S(n)). \quad (4.2)$$

Proof. Consider the binary decision tree associated with an arbitrary correct deterministic algorithm for problem P . Let t denote the height of the tree, representing the maximum length of any root-to-leaf path.

Because comparisons are strictly binary (assuming distinct keys), every internal node has at most 2 children. By induction on height, a binary tree of height t possesses at most L leaves, bounded by:

$$L \leq 2^t. \quad (4.3)$$

For the algorithm to be correct on every valid input instance, the decision tree must be capable of reaching each of the $S(n)$ distinct solutions. Consequently,

each distinct solution must be associated with at least one distinct leaf in the tree (if two distinct solutions mapped to the same leaf, the algorithm would fail to discriminate between them, outputting an erroneous answer for at least one of the instances). Hence, we must have:

$$2^t \geq L \geq S(n). \quad (4.4)$$

Applying the base-2 logarithm (a strictly monotonically increasing function) to both sides yields:

$$t \geq \log_2 S(n). \quad (4.5)$$

Since the worst-case time complexity $T(n)$ is defined as the height t of the tree, we conclude that:

$$T(n) \geq \log_2 S(n) \implies T(n) = \Omega(\log_2 S(n)). \quad (4.6)$$

□

4.1.3 Fundamental Applications: Searching and Sorting

[Theorem 4.1.1](#) directly yields canonical lower bounds for two core computational problems in internal memory:

1. **Searching a Key (*Searching*):** Given a sorted array of n elements and a target key K , the algorithm must determine whether K resides in the array and, if so, return its index $i \in \{1, 2, \dots, n\}$; if absent, it returns an explicit failure symbol $\perp$. The set of distinct valid outputs is:

$$S(n) = \{1, 2, \dots, n\} \cup \{\perp\} \implies |S(n)| = n + 1. \quad (4.7)$$

Applying the theorem:

$$T_{\text{Search}}(n) \geq \log_2(n + 1) = \Omega(\log_2 n). \quad (4.8)$$

This lower bound establishes the asymptotic optimality of *Binary Search*, which operates in exactly $\Theta(\log_2 n)$ comparisons.

2. **Sorting (*Sorting*):** Given an array of n distinct elements, sorting is equivalent to identifying which of the $n!$ permutations of indices rearranges the keys in non-decreasing order. Since the set $\{1, 2, \dots, n\}$ admits exactly $n!$ distinct permutations, the number of possible solutions is:

$$S(n) = n!. \quad (4.9)$$

Applying the lower bound theorem:

$$T_{\text{Sort}}(n) \geq \log_2(n!). \quad (4.10)$$

To express this bound in explicit asymptotic form, we invoke **Stirling's approximation** for the factorial:

$$n! \approx \sqrt{2\pi n} \left(\frac{n}{e}\right)^n \implies \ln(n!) = n \ln n - n + \mathcal{O}(\ln n). \quad (4.11)$$

Converting to base-2 logarithm:

$$\log_2(n!) = n \log_2 n - n \log_2 e + \Theta(\log_2 n) = \Theta(n \log_2 n). \quad (4.12)$$

Alternatively, using an elementary lower bound over the upper half of terms:

$$n! \geq \prod_{i=\lceil n/2 \rceil}^n i \geq \left(\frac{n}{2}\right)^{n/2}, \quad (4.13)$$

$$\log_2(n!) \geq \frac{n}{2} \log_2 \left(\frac{n}{2}\right) = \frac{n}{2}(\log_2 n - 1) = \Omega(n \log_2 n). \quad (4.14)$$

Therefore:

$$T_{\text{Sort}}(n) = \Omega(n \log_2 n). \quad (4.15)$$

Any deterministic or randomized comparison-based sorting algorithm (such as *Merge Sort*, *Heap Sort*, or the expected time of *Quicksort*) cannot surpass the asymptotic barrier $\Omega(n \log_2 n)$ in the RAM model.

4.2 Two-Level Memory: Aggarwal-Vitter Lower Bound

In the two-level memory model introduced by Aggarwal and Vitter [2], the primary computational cost is not measured by elementary CPU comparisons, but by the total number t of I/O operations, each transferring a contiguous block of B elements between slow external storage and internal RAM of capacity M items.

4.2.1 The Nature of I/O and Free In-Memory Comparisons

The standard binary decision tree cannot be directly applied to external memory analysis:

- A single disk read operation simultaneously brings an entire block of B elements into fast internal memory.
- Once these B elements reside in RAM alongside the remaining $M - B$ elements already present, the CPU can execute **an arbitrary number of in-memory comparisons at zero I/O cost**.
- These "free" internal comparisons permit sorting all M items currently in RAM, drastically pruning the set of candidate global permutations consistent with the current knowledge.

Consequently, to establish an external memory lower bound, one must formulate an **I/O Decision Tree**, where each node represents an I/O read operation (loading a block of size B), and outgoing edges correspond to all possible relative arrangements resulting from interleaving the B retrieved elements with the $M - B$ elements already residing in RAM.

4.2.2 Branching Factor (Fan-out) Calculation of the I/O Tree

When a block of B elements is transferred from disk into internal memory (accommodating M elements total, consisting of $M - B$ pre-existing elements and B newly loaded items), we distinguish two operational scenarios:

1. **Reading a "New Block" (*New Block*):** The block is transferred from disk for the very first time: its elements have never been observed before by any previous step of the algorithm.
 - The mutual relative order of the B elements is completely unknown: they can appear in any of the $B!$ possible permutations.
 - Once placed in RAM, the B new items must be interleaved with the $M - B$ pre-existing items to establish their combined sorted order within the M slots. The number of ways to interleave B elements into M positions is given by the binomial coefficient:

$$\binom{M}{B} = \frac{M!}{B!(M-B)!}. \quad (4.16)$$

Multiplying the $B!$ internal ordering possibilities by the $\binom{M}{B}$ interleaving positions, the branching factor (*fan-out*) X for reading a new block is:

$$X = (B!) \cdot \binom{M}{B}. \quad (4.17)$$

2. **Re-reading an "Old Block" (*Old Block*):** The block was previously read, internally sorted, and written back to disk during an earlier phase of computation (such as an intermediate run generated during multiway merging).

- The internal relative order among its B elements is already fully known; hence, no new information is acquired regarding their intrinsic ordering (a factor of 1 rather than $B!$).
- The only new information gained is how these B elements interleave with the other $M - B$ items currently in RAM.

The branching factor (*fan-out*) Y for re-reading an old block is therefore:

$$Y = \binom{M}{B}. \quad (4.18)$$

Figure 4.2 summarizes the logical dichotomy and branch cardinalities for both I/O transfer types.

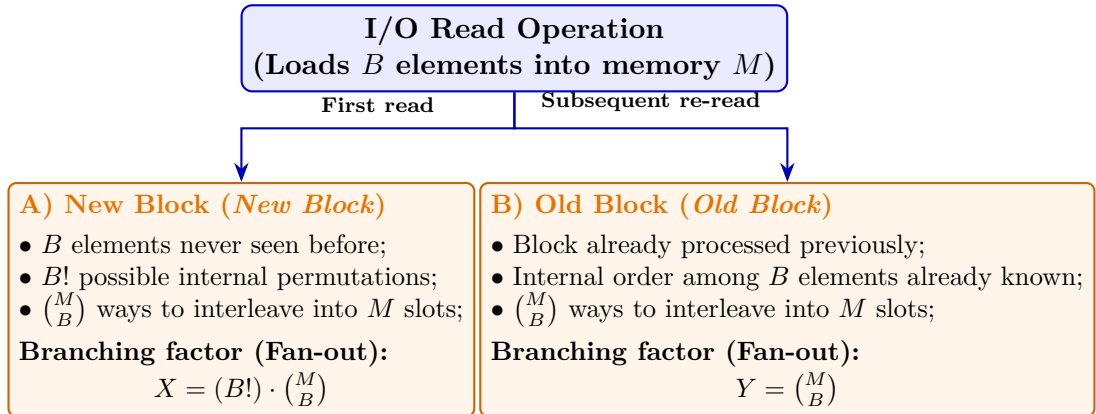


Figure 4.2: Branching factors (*fan-out*) in the I/O decision tree under the two-level memory model: reading a new block yields internal order information ($B!$) and interleaving information ($\binom{M}{B}$); re-reading an old block yields only interleaving information.

4.2.3 Formal Derivation of the Aggarwal-Vitter Lower Bound

Consider an input file consisting of n distinct elements stored across $N = \frac{n}{B}$ disk blocks. In order to sort the file, any correct algorithm must inspect each

element at least once; consequently, it must execute at least $\frac{n}{B}$ "New Block" read operations.

If the algorithm executes a total of t I/O operations in the worst case, then:

- Exactly $\frac{n}{B}$ operations will be of type "New Block", each with fan-out at most X .
- The remaining $t - \frac{n}{B}$ operations will be (at most) of type "Old Block", each with fan-out at most Y .

The total number of leaves in the I/O decision tree represents the maximum number of global permutations distinguishable by this sequence of transfers. For the algorithm to correctly sort any valid instance, this leaf count must be at least equal to $n!$:

$$X^{\frac{n}{B}} \cdot Y^{(t - \frac{n}{B})} \geq n!. \quad (4.19)$$

Substituting the explicit formulas for X and Y from equations (4.17) and (4.18):

$$\left[(B!) \cdot \binom{M}{B} \right]^{\frac{n}{B}} \cdot \left[\binom{M}{B} \right]^{t - \frac{n}{B}} \geq n!. \quad (4.20)$$

Collecting powers sharing the common base $\binom{M}{B}$:

$$(B!)^{\frac{n}{B}} \cdot \binom{M}{B}^{\frac{n}{B}} \cdot \binom{M}{B}^{t - \frac{n}{B}} \geq n! \implies (B!)^{\frac{n}{B}} \cdot \binom{M}{B}^t \geq n!. \quad (4.21)$$

Taking the base-2 logarithm on both sides of inequality (4.21):

$$\log_2 \left((B!)^{\frac{n}{B}} \right) + \log_2 \left(\binom{M}{B}^t \right) \geq \log_2(n!), \quad (4.22)$$

which yields:

$$\frac{n}{B} \log_2(B!) + t \log_2 \left(\binom{M}{B} \right) \geq \log_2(n!). \quad (4.23)$$

To expand the logarithmic expressions, we use standard asymptotic bounds for the factorial and the binomial coefficient:

- By Stirling's approximation: $\log_2(a!) \approx a \log_2 a - a \log_2 e \approx a \log_2 a$.
- For the binomial coefficient, recalling that $\binom{a}{b} \leq \left(\frac{e \cdot a}{b} \right)^b$:

$$\log_2 \left(\binom{M}{B} \right) \approx B \log_2 \left(\frac{M}{B} \right) + \mathcal{O}(B). \quad (4.24)$$

Substituting these expressions into equation (4.23):

$$\frac{n}{B} \cdot (B \log_2 B) + t \cdot \left(B \log_2 \frac{M}{B} \right) \geq n \log_2 n. \quad (4.25)$$

Simplifying the factor B in the first term:

$$n \log_2 B + t \cdot B \log_2 \left(\frac{M}{B} \right) \geq n \log_2 n. \quad (4.26)$$

Isolating the term containing the unknown t :

$$t \cdot B \log_2 \left(\frac{M}{B} \right) \geq n \log_2 n - n \log_2 B = n (\log_2 n - \log_2 B) = n \log_2 \left(\frac{n}{B} \right). \quad (4.27)$$

Dividing both sides by the strictly positive quantity $B \log_2 \left(\frac{M}{B} \right)$:

$$t \geq \frac{n \log_2 \left(\frac{n}{B} \right)}{B \log_2 \left(\frac{M}{B} \right)} = \frac{n}{B} \cdot \frac{\log_2(n/B)}{\log_2(M/B)}. \quad (4.28)$$

Applying the change of base formula for logarithms ($\frac{\log_2 a}{\log_2 b} = \log_b a$), we formally obtain the renowned Aggarwal-Vitter lower bound theorem.

Theorem 4.2.1 (Aggarwal-Vitter Lower Bound for External Sorting). *Under the two-level memory model, any deterministic or randomized comparison-based sorting algorithm requires in the worst case:*

$$I/O_{Sort}(n) = \Omega \left(\frac{n}{B} \log_{M/B} \left(\frac{n}{B} \right) \right) = \Omega \left(\frac{n}{B} \frac{\log(n/B)}{\log(M/B)} \right) \text{ I/Os.} \quad (4.29)$$

Property 4.2.1 (Asymptotic Optimality of Multiway Merge Sort). *The Aggarwal-Vitter lower bound demonstrates that the **Multiway Merge Sort** (k -Way Merge Sort with merge degree $k = \frac{M}{B} - 1$) analyzed in Lecture 3 is **asymptotically optimal**: the total number of I/O operations required to sort the array matches the fundamental theoretical lower bound dictated by information theory in external memory.*

4.3 Quicksort: Duplicate Handling and Dual-Pivot Quicksort

Quicksort is a *Divide and Conquer* algorithm based on distribution (*distribution-based*): rather than rigidly partitioning the array by index positions (as in *Merge Sort*) and then merging the sorted subproblems, Quicksort partitions the data based on the relative magnitude of one or more privileged keys termed **pivots**, delegating all sorting work to the partitioning phase.

4.3.1 The Equal Values Problem

In traditional textbook implementations of Quicksort employing standard two-way partitioning (*2-way partitioning*, such as Lomuto or Hoare schemes):

- The array is divided into two parts: elements $\leq p$ on one side and elements $> p$ on the other.
- Whenever the array contains many identical keys equal to pivot p (in the extreme case, an array containing only identical keys, $S[i] = c, \forall i$), such a scheme places all duplicates into the same partition.
- As a consequence, the subproblem shrinks by only a single element per recursive call (1 versus $n - 1$), collapsing the recursion tree into a degenerate chain of height $\Theta(n)$ and the overall running time to quadratic complexity:

$$T(n) = T(n - 1) + \Theta(n) = \Theta(n^2). \quad (4.30)$$

Definition 4.3.1 (Three-Way Partitioning (Dutch National Flag)). The classical remedy for duplicate keys, formulated by Edsger Dijkstra [3] as the Dutch National Flag problem, partitions the array into **three contiguous regions**:

$$[< p \quad | \quad = p \quad | \quad > p]. \quad (4.31)$$

Elements strictly equal to the pivot ($= p$) are already in their final sorted positions and are **excluded from further recursive invocations**, which are executed solely on the $[< p]$ and $[> p]$ subranges. On arrays with all equal keys, this achieves linear running time $\mathcal{O}(n)$.

4.3.2 3-Way Partitioning Algorithm and Numerical Execution Example

To practically implement three-way partitioning (*Dutch National Flag*) in Quicksort, the linear-time algorithm designed by Dijkstra and formalized by Sedgewick [4] is widely adopted.

Given a sub-array $S[\text{lo} \dots \text{hi}]$ with the first element chosen as the pivot $p = S[\text{lo}]$, the algorithm maintains three operational indices during sequential scanning:

- **Pointer lt (*less than*)**: delimits elements strictly smaller than the pivot on the left. The invariant guarantees that $S[k] < p$ for all $\text{lo} \leq k < lt$. Initially, $lt = \text{lo}$.

- **Pointer i (scan cursor):** scans the array elements. The invariant ensures that $S[k] = p$ for all $lt \leq k < i$. Initially, $i = lo + 1$.
- **Pointer gt (*greater than*):** delimits elements strictly greater than the pivot on the right. The invariant guarantees that $S[k] > p$ for all $gt < k \leq hi$. Initially, $gt = hi$.
- The sub-array $S[i \dots gt]$ contains elements that have not yet been examined.

At each iteration, as long as $i \leq gt$, the current element $S[i]$ is compared with the pivot p :

1. **If $S[i] < p$:** swap $S[lt]$ and $S[i]$, then increment both indices ($lt \leftarrow lt + 1$, $i \leftarrow i + 1$). Because by invariant cell $S[lt]$ held an element equal to p , swapping moves the smaller element to the left and places a pivot duplicate at $S[i]$, allowing i to advance.
2. **If $S[i] > p$:** swap $S[i]$ and $S[gt]$, then decrement only gt ($gt \leftarrow gt - 1$). The index i **does not advance**, because the element just swapped from $S[gt]$ has not yet been compared.
3. **If $S[i] == p$:** the element is already in the correct central region; increment only the scan cursor ($i \leftarrow i + 1$).

Upon termination ($i > gt$), the array is partitioned into three distinct contiguous regions: $S[lo \dots lt - 1] < p$, $S[lt \dots gt] = p$, and $S[gt + 1 \dots hi] > p$. Subsequent recursive calls are issued **exclusively** on $S[lo \dots lt - 1]$ and $S[gt + 1 \dots hi]$, bypassing the entire duplicate block.

Figure 4.3 depicts the invariant structure and the visual execution trace on an example array with pivot $p = 4$.

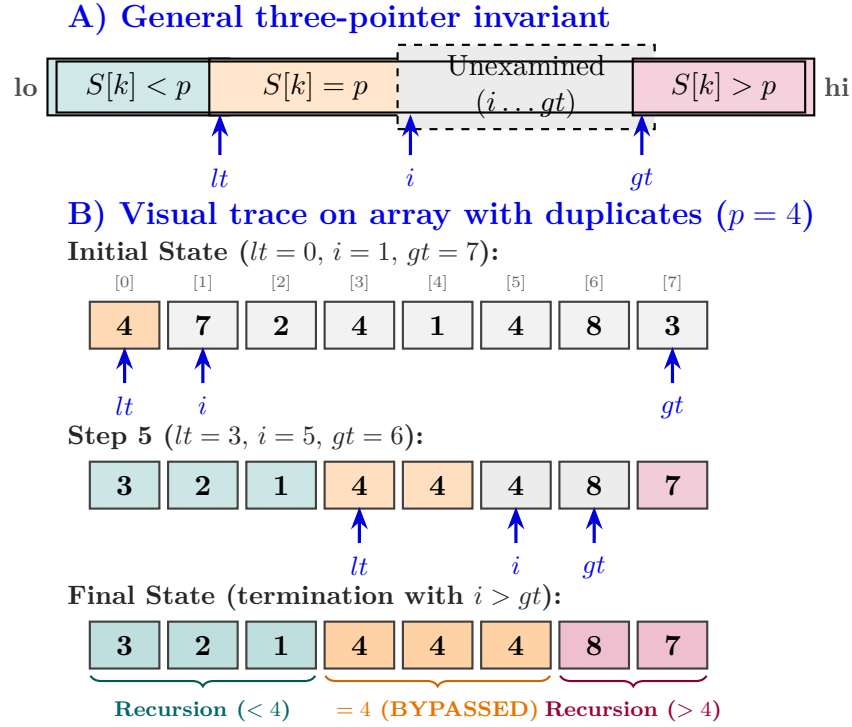


Figure 4.3: Dijkstra’s three-way partitioning (*Dutch National Flag*): A) Invariant of the three pointers lt, i, gt . B) Visual execution trace on an array with duplicate elements with pivot $p = 4$. The middle region containing all duplicate keys is placed in its final position and bypassed by recursive calls.

Table 4.1 details the step-by-step progress of the three pointers and array contents throughout the execution.

Step	i	lt	gt	Comparison	Action	Array State
Init	1	0	7	Pivot $p = 4$	Initialization	[4, 7, 2, 4, 1, 4, 8, 3]
1	1	0	7	$S[1] = 7 > 4$	SWAP($S[1], S[7]$), $gt \leftarrow 6$	[4, 3, 2, 4, 1, 4, 8, 7]
2	1	0	6	$S[1] = 3 < 4$	SWAP($S[0], S[1]$), $lt \leftarrow 1, i \leftarrow 2$	[3, 4, 2, 4, 1, 4, 8, 7]
3	2	1	6	$S[2] = 2 < 4$	SWAP($S[1], S[2]$), $lt \leftarrow 2, i \leftarrow 3$	[3, 2, 4, 4, 1, 4, 8, 7]
4	3	2	6	$S[3] == 4$	Advance $i \leftarrow 4$	[3, 2, 4, 4, 1, 4, 8, 7]
5	4	2	6	$S[4] = 1 < 4$	SWAP($S[2], S[4]$), $lt \leftarrow 3, i \leftarrow 5$	[3, 2, 1, 4, 4, 4, 8, 7]
6	5	3	6	$S[5] == 4$	Advance $i \leftarrow 6$	[3, 2, 1, 4, 4, 4, 8, 7]
7	6	3	6	$S[6] = 8 > 4$	SWAP($S[6], S[6]$), $gt \leftarrow 5$	[3, 2, 1, 4, 4, 4, 8, 7]
End	6	3	5	$i > gt$	Termination	[3, 2, 1 4, 4, 4 8, 7]

Table 4.1: Detailed step-by-step trace of 3-way partitioning with pivot $p = 4$. At termination, the three identical keys equal to 4 occupy indices $lt \dots gt$ (3...5) and are bypassed by subsequent recursive calls.

4.3.3 Architecture of Dual-Pivot Quicksort (Yaroslavskiy)

In 2009, Russian engineer Vladimir Yaroslavskiy proposed an innovative algorithm called **Dual-Pivot Quicksort** [5]. Following extensive empirical benchmarks conducted in collaboration with Joshua Bloch and Robert Sedgewick, the algorithm replaced classic single-pivot Quicksort in Java’s standard library (‘java.util.Arrays.sort’) for all primitive data types.

Unlike classical Quicksort, Dual-Pivot Quicksort chooses **two pivots** P and Q . It first ensures $P \leq Q$ (swapping them if necessary). The active sub-array range $S[\text{left} \dots \text{right}]$ is dynamically divided into four regions managed via three operational indices (l, k, g):

- l (*less*): upper bound of elements strictly smaller than P ($S[i] < P$ for all $\text{left} \leq i < l$).
- k : current scanning pointer ($l \leq k \leq g$). Elements between l and $k - 1$ satisfy the central invariant: $P \leq S[i] \leq Q$.
- g (*great*): lower bound of elements strictly greater than Q ($S[i] > Q$ for all $g < i \leq \text{right}$).
- The region between k and g holds items not yet examined.

Figure 4.4 illustrates the array layout during the execution of dual-pivot partitioning.

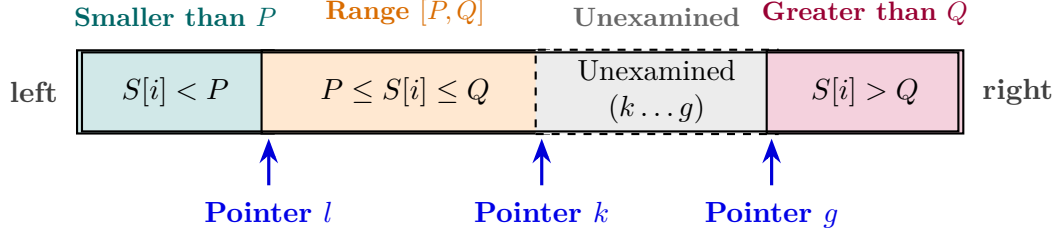


Figure 4.4: Dual-Pivot Quicksort partitioning schema (Yaroslavskiy). The array is partitioned into four dynamic regions: elements $< P$, elements between P and Q , unexamined elements between k and g , and elements $> Q$.

4.3.4 Invariants and Transition Logic

The partitioning procedure maintains the invariant until pointer k crosses g ($k \leq g$). At each step, the element $S[k]$ is evaluated:

1. **Case 1:** $S[k] < P$: The element belongs to the leftmost partition. It is swapped with $S[l]$, and both pointers are incremented:

$$\text{SWAP}(S[k], S[l]), \quad l \leftarrow l + 1, \quad k \leftarrow k + 1. \quad (4.32)$$

Since by invariant the element formerly at $S[l]$ was within the middle range $[P, Q]$, the element moved to position k is guaranteed to belong to $[P, Q]$, allowing k to safely advance.

2. **Case 2:** $S[k] > Q$: The element belongs to the rightmost partition. Before swapping, pointer g is moved backward while $S[g] > Q$ (skipping elements already correctly placed on the right). Once an element $S[g] \leq Q$ is found, a swap is performed:

$$\text{SWAP}(S[k], S[g]), \quad g \leftarrow g - 1. \quad (4.33)$$

The element now at $S[k]$ (retrieved from g) has not yet been compared against P ; hence it is **re-examined on the next step without advancing k** .

3. **Case 3:** $P \leq S[k] \leq Q$: The element is already in its proper position in the middle partition. No swap is required, and k simply advances:

$$k \leftarrow k + 1. \quad (4.34)$$

When scanning terminates ($k > g$), the two pivots P and Q are swapped into their final positions $l - 1$ and $g + 1$, respectively. Three independent recursive calls are then launched on the three remaining subranges:

$$[\text{left} \dots l - 2], \quad [l \dots g], \quad [g + 2 \dots \text{right}]. \quad (4.35)$$

Algorithm 3: Dual-Pivot Partitioning (Yaroslavskiy)

Input: Array S , start index left , end index right with $\text{left} < \text{right}$

Output: Final pivot positions (p_1, p_2)

```

1 if  $S[\text{left}] > S[\text{right}]$  then
2    $\text{SWAP}(S[\text{left}], S[\text{right}])$ 
3  $P \leftarrow S[\text{left}]$ 
4  $Q \leftarrow S[\text{right}]$ 
5  $l \leftarrow \text{left} + 1$ 
6  $g \leftarrow \text{right} - 1$ 
7  $k \leftarrow l$ 
8 while  $k \leq g$  do
9   if  $S[k] < P$  then
10     $\text{SWAP}(S[k], S[l])$ 
11     $l \leftarrow l + 1$ 
12     $k \leftarrow k + 1$ 
13   else if  $S[k] > Q$  then
14     while  $S[g] > Q$  and  $k < g$  do
15        $g \leftarrow g - 1$ 
16      $\text{SWAP}(S[k], S[g])$ 
17      $g \leftarrow g - 1$ 
18     if  $S[k] < P$  then
19        $\text{SWAP}(S[k], S[l])$ 
20        $l \leftarrow l + 1$ 
21      $k \leftarrow k + 1$ 
22   else
23      $k \leftarrow k + 1$ 
24  $l \leftarrow l - 1$ 
25  $g \leftarrow g + 1$ 
26  $\text{SWAP}(S[\text{left}], S[l])$ 
27  $\text{SWAP}(S[\text{right}], S[g])$ 
28 return  $(l, g)$ 

```

4.3.5 Execution Example and Trace of Dual-Pivot Quicksort

To illustrate the mechanics of the dual-pivot partitioning scheme described in [algorithm 3](#), consider an array of 8 elements:

$$S = [3, 8, 2, 5, 1, 4, 9, 7]. \quad (4.36)$$

Pivots are selected from the boundary positions: $P = S[0] = 3$ and $Q = S[7] = 7$. Since $P \leq Q$ ($3 \leq 7$), the preliminary pivot ordering condition holds. The cursors are initialized as: $l = 1$, $k = 1$, and $g = 6$.

In [Figure 4.5](#), key snapshots of the execution are shown, illustrating the progression of pointers l, k, g and the final swap of P and Q .

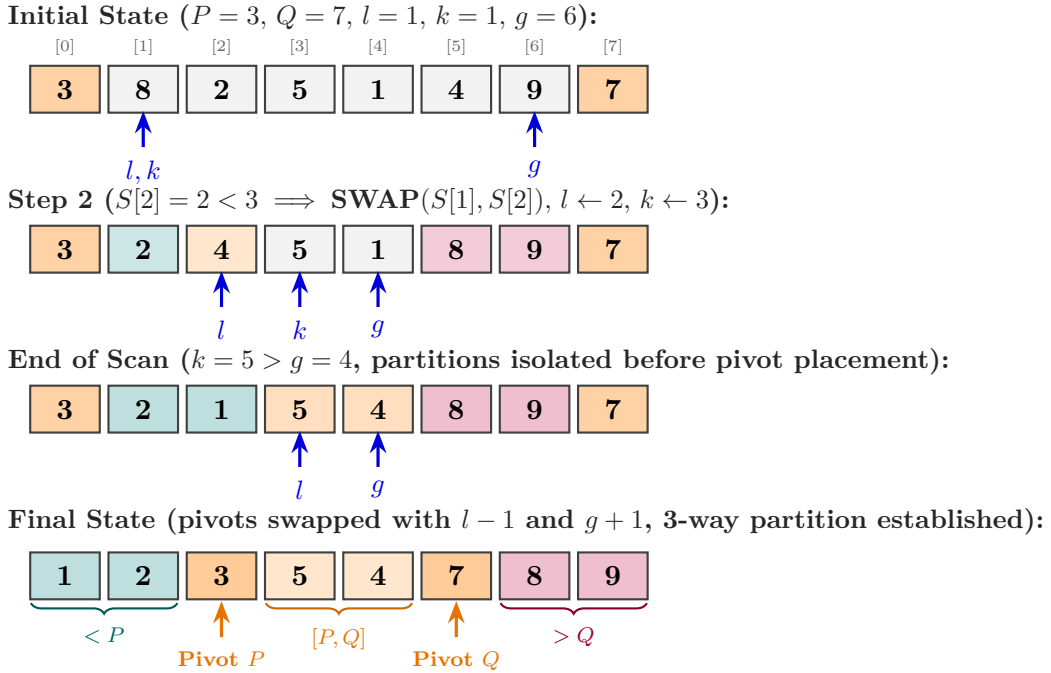


Figure 4.5: Execution trace of Dual-Pivot partitioning (Yaroslavskiy’s scheme) on an array with $P = 3$ and $Q = 7$. After the linear scan terminates, the pivots are swapped into their final positions (indices 2 and 5), isolating the three subproblems for subsequent recursive calls.

In [Table 4.2](#), the step-by-step operation sequence executed by Dual-Pivot Quicksort is summarized.

Step	k	l	g	Element	Action	Array State
Init	1	1	6	Pivots $P = 3, Q = 7$	Initialize cursors	[3, 8, 2, 5, 1, 4, 9, 7]
1	1	1	6	$S[1] = 8 > 7$	Find $g = 5$; SWAP($S[1], S[5]$), $g \leftarrow 4, k \leftarrow 2$	[3, 4, 2, 5, 1, 8, 9, 7]
2	2	1	4	$S[2] = 2 < 3$	SWAP($S[1], S[2]$), $l \leftarrow 2, k \leftarrow 3$	[3, 2, 4, 5, 1, 8, 9, 7]
3	3	2	4	$3 \leq S[3] = 5 \leq 7$	Advance $k \leftarrow 4$	[3, 2, 4, 5, 1, 8, 9, 7]
4	4	2	4	$S[4] = 1 < 3$	SWAP($S[2], S[4]$), $l \leftarrow 3, k \leftarrow 5$	[3, 2, 1, 5, 4, 8, 9, 7]
End	5	3	4	$k > g$	Stop scan	[3, 2, 1, 5, 4, 8, 9, 7]
Pivot	–	2	5	Relocation	SWAP($S[0], S[2]$), SWAP($S[7], S[5]$)	[1, 2 3 5, 4 7 8, 9]

Table 4.2: Step-by-step trace of Dual-Pivot partitioning with pivots $P = 3$ and $Q = 7$. Upon termination, both pivots reside at their definitive index locations (2 and 5) and recursive calls process the three sub-regions independently.

Property 4.3.1 (Architectural Benefits of Dual-Pivot Quicksort). *Deploying two pivots provides substantial algorithm engineering advantages:*

- **Reduced Recursion Tree Depth:** *Dividing the array into 3 sub-problems instead of 2 reduces the average recursion tree height from $\log_2 n \approx 1.44 \ln n$ down to $\log_3 n \approx 0.91 \ln n$, directly lowering the total number of memory scans.*
- **Cache Efficiency and CPU Pipelining:** *While the theoretical comparison count increases slightly compared to single-pivot Quicksort ($1.9n \ln n$ vs $2n \ln n$), the total number of cache line invalidations and memory writes drops significantly, making superior use of modern memory hierarchies and CPU pipeline architectures.*

4.4 Bounded Quicksort (BQS) and Tail Recursion Elimination

Despite its optimal expected time complexity $\mathcal{O}(n \log_2 n)$, standard recursive Quicksort exhibits a severe vulnerability concerning auxiliary memory usage on the call stack.

4.4.1 Call Stack Vulnerability

Upon each recursive invocation, the runtime environment allocates a call stack frame containing function parameters, local variables, and the return address:

- In balanced or near-optimal partitioning scenarios, the call stack depth is $\Theta(\log_2 n)$, consuming negligible extra space.
- However, under severely unbalanced partitions (worst-case scenario: pivot located at an extreme, yielding 1 versus $n - 1$), stack depth grows linearly with input size:

$$\text{Stack Depth} = \Theta(n). \quad (4.37)$$

For large inputs ($n \geq 10^6$), linear stack growth inevitably causes an unrecoverable system abort due to a *Stack Overflow Exception*.

4.4.2 Bounded Quicksort Algorithm (BQS)

Bounded Quicksort (BQS) resolves this critical design flaw through two established software engineering principles:

1. **Tail Recursion Elimination (*Tail Call Elimination*):** The algorithm never performs two recursive calls within the same activation frame. It invokes recursion **strictly on the partition of size at most half the current subproblem span** ($\leq \frac{j-i}{2}$). The remaining partition (the larger half) is handled iteratively by updating indices i or j inside an enclosing ‘while’ loop.
2. **Insertion Sort Cutoff for Small Subproblems:** When the subarray length falls below a predetermined threshold m_0 (typically $m_0 \in [16, 32]$), recursion ceases and sorting is completed using insertion sort.

Algorithm 4: Bounded Quicksort (BQS)

Input: Array S , start index i , end index j , cutoff threshold m_0

Output: Sorted subarray $S[i \dots j]$

```
1 while  $(j - i) \geq m_0$  do
2    $r \leftarrow \text{RANDOM\_POSITION}(i, \dots, j)$ 
3    $\text{SWAP}(S[r], S[i])$  // Random pivot moved to the beginning
4    $p' \leftarrow \text{PARTITION}(S, i, j)$  //  $p'$  is the final pivot
   position
5   if  $p' \leq \lfloor (i + j)/2 \rfloor$  then
6     // LEFT partition is smaller or equal: recurse
       $\text{BQS}(S, i, p' - 1, m_0)$ 
      // Tail recursion elimination on the RIGHT:
      iterate
7      $i \leftarrow p' + 1$ 
8   else
9     // RIGHT partition is smaller or equal: recurse
       $\text{BQS}(S, p' + 1, j, m_0)$ 
      // Tail recursion elimination on the LEFT: iterate
10     $j \leftarrow p' - 1$ 
11  $\text{INSERTION\_SORT}(S, i, j)$  // Process small base cases of
    size  $< m_0$ 
```

Figure 4.6 depicts the operational control flow of BQS, highlighting the branch between bounded recursion and iterative continuation.

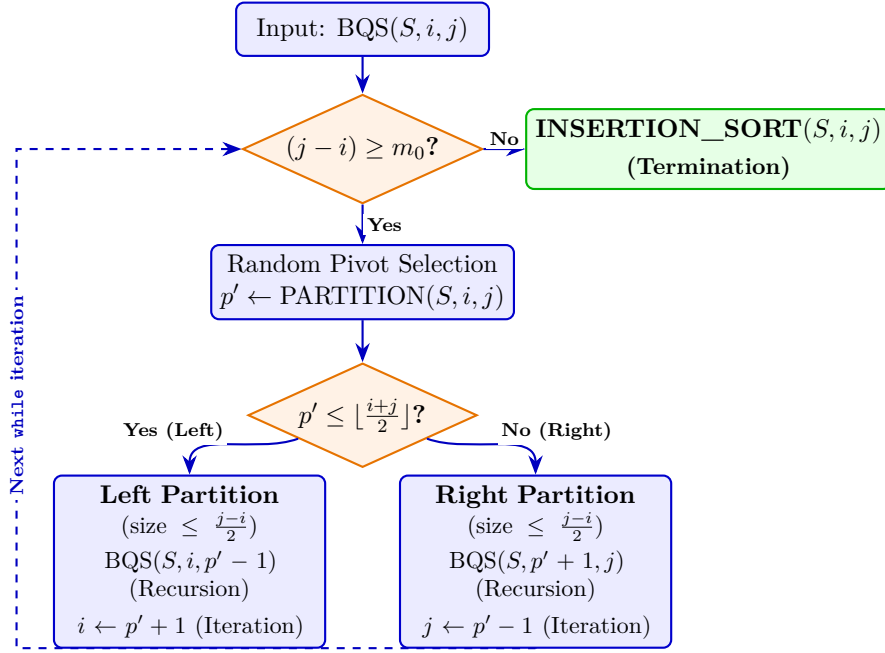


Figure 4.6: Control flow of Bounded Quicksort (BQS). Recursive invocations are strictly bounded to the smaller partition ($\leq (j - i)/2$), while the larger half is evaluated iteratively inside the **while** loop. Partitions smaller than m_0 are sorted directly via Insertion Sort.

4.4.3 Execution Example and Call Stack Management in Bounded Quicksort

To gain an in-depth understanding of the stack bounding mechanism and tail call elimination, consider the execution of [algorithm 4](#) on an 8-element array with cutoff threshold $m_0 = 3$:

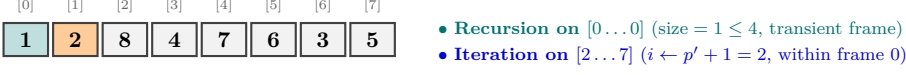
$$S = [7, 2, 8, 4, 1, 6, 3, 5]. \quad (4.38)$$

In an unfavorable scenario where partitions are repeatedly skewed, standard Quicksort pushes a new call frame for each subproblem, driving stack depth linearly to $\Theta(n)$. Conversely, Bounded Quicksort processes the larger subproblem purely iteratively within the existing stack frame (by updating boundary index i or j), invoking recursion exclusively on the subarray whose size does not exceed half the current range ($\leq L/2$).

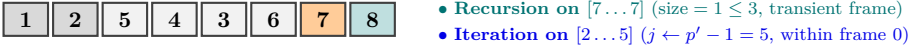
In [Figure 4.7](#), the execution sequence and the structural comparison between the call stack of standard Quicksort and Bounded Quicksort are illustrated.

A. Main While-Loop Iteration Sequence in BQS ($m_0 = 3$)

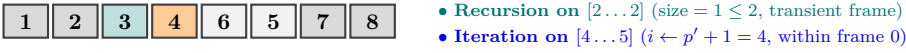
Iteration 1: Range $[0 \dots 7]$ ($L = 8$). Pivot $p' = 1$ (value 2). Test: $p' = 1 \leq \lfloor 7/2 \rfloor = 3$ (**True**)



Iteration 2: Range $[2 \dots 7]$ ($L = 6$). Pivot $p' = 6$ (value 7). Test: $p' = 6 \leq \lfloor 9/2 \rfloor = 4$ (**False**)



Iteration 3: Range $[2 \dots 5]$ ($L = 4$). Pivot $p' = 3$ (value 4). Test: $p' = 3 \leq \lfloor 7/2 \rfloor = 3$ (**True**)



Base Cutoff: Range $[4 \dots 5]$ ($L = 2 < m_0 = 3$). while loop terminates



B. Call Stack Structure: Standard Quicksort vs Bounded Quicksort

Standard Quicksort (Linear Stack $\Theta(n)$) **Bounded Quicksort (Logarithmic Stack $\leq \lfloor \log_2 n \rfloor$):**

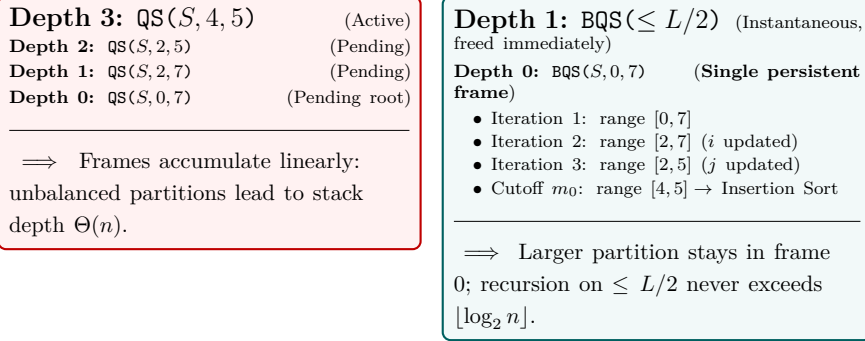


Figure 4.7: Execution trace of Bounded Quicksort (BQS) with threshold $m_0 = 3$ and call stack memory comparison. **(A)** At each iteration, the larger partition is processed within the same stack frame by updating i or j , while only the halved partition ($\leq L/2$) triggers a short-lived recursive call. **(B)** Standard Quicksort accumulates call frames linearly; BQS strictly guarantees logarithmic stack depth.

In Table 4.3, the step-by-step iteration sequence is detailed, highlighting the smaller-partition test, the tail recursion elimination update, and the instantaneous maximum stack depth.

Step	$[i, j]$	L	Pivot	Test	Action (Recursion & Iteration)	Array State	Stack
Init	$[0, 7]$	8	–		Root frame BQS($S, 0, 7$)	$[7, 2, 8, 4, 1, 6, 3, 5]$	0
1	$[0, 7]$	8	$p' = 1 \leq 3$		Rec: $[0, 0]$ (size ≤ 4 , instant return) Iter: $i \leftarrow 2$ on $[2, 7]$ (same frame)	$[1 \mid \mathbf{2} \mid 8, 4, 7, 6, 3, 5]$	1
2	$[2, 7]$	6	$p' = 6 > 4$		Rec: $[7, 7]$ (size ≤ 3 , instant return) Iter: $j \leftarrow 5$ on $[2, 5]$ (same frame)	$[1, 2 \mid 5, 4, 3, 6 \mid \mathbf{7} \mid 8]$	1
3	$[2, 5]$	4	$p' = 3 \leq 3$		Rec: $[2, 2]$ (size ≤ 2 , instant return) Iter: $i \leftarrow 4$ on $[4, 5]$ (same frame)	$[1, 2 \mid 3 \mid \mathbf{4} \mid 6, 5 \mid 7 \mid 8]$	1
Cutoff	$[4, 5]$	2	$L < m_0$		Exit: Exit <code>while</code> loop Sort: Insertion Sort on $[4, 5]$	$[1, 2, 3, 4, \mathbf{5}, \mathbf{6}, 7, 8]$	0

Table 4.3: Step-by-step trace of Bounded Quicksort (BQS) with cutoff threshold $m_0 = 3$. In each iteration, the subproblem of size $\leq L/2$ is resolved with a bounded recursive call, while the larger subproblem is processed iteratively within the current frame, bounding stack depth to ≤ 1 in this trace ($\leq \lfloor \log_2 n \rfloor$ in general).

4.4.4 Rigorous Analysis of the $\Theta(\log_2 n)$ Extra Space Bound

The cornerstone of BQS lies in its absolute mathematical guarantee on memory consumption:

Theorem 4.4.1 (Auxiliary Space Guarantee of Bounded Quicksort). *For any input of size n , Bounded Quicksort consumes an auxiliary call stack space strictly bounded by:*

$$\text{Auxiliary Space} = \Theta(\log_2 n). \quad (4.39)$$

The maximum stack depth never exceeds $\lfloor \log_2 n \rfloor$ under any worst-case scenario.

Proof. Let $L = j - i + 1$ denote the length of the active subarray at the entry of a BQS call. Upon partitioning around index p' , the algorithm inspects the lengths of the two subranges:

- The left subrange has length $L_{\text{left}} = (p' - 1) - i + 1 = p' - i$.
- The right subrange has length $L_{\text{right}} = j - (p' + 1) + 1 = j - p'$.

The condition $p' \leq \lfloor \frac{i+j}{2} \rfloor$ implies:

$$p' - i \leq \frac{j - i}{2} < \frac{L}{2}. \quad (4.40)$$

Consequently, when this condition holds, recursion is invoked exclusively on the left partition, whose size is strictly bounded by:

$$L_{\text{recursive}} \leq \left\lfloor \frac{L}{2} \right\rfloor. \quad (4.41)$$

Conversely, when $p' > \lfloor \frac{i+j}{2} \rfloor$, the right subrange satisfies $j - p' < \frac{L}{2}$, and the recursive call is invoked on the right partition, fulfilling the exact same inequality.

The remaining subproblem has size $L_{\text{iterative}} \geq \frac{L}{2}$, but **does not spawn any recursive call**: the algorithm simply mutates $i \leftarrow p' + 1$ or $j \leftarrow p' - 1$ and loops within the existing activation frame without allocating any new stack space.

Therefore, the input size passed to any subsequent stack frame decreases by at least a factor of 2 relative to its parent frame:

$$L^{(k)} \leq \frac{n}{2^k}, \quad (4.42)$$

where k denotes the current stack depth. The recursion can continue at most while $L^{(k)} \geq m_0 \geq 1$:

$$\frac{n}{2^k} \geq 1 \implies 2^k \leq n \implies k \leq \lfloor \log_2 n \rfloor. \quad (4.43)$$

Since each activation record requires constant memory $\mathcal{O}(1)$, the total auxiliary stack space is strictly bounded by:

$$\text{Extra Space} = \mathcal{O}(1) \cdot \lfloor \log_2 n \rfloor = \Theta(\log_2 n). \quad (4.44)$$

□

4.4.5 Hybrid Optimization with Insertion Sort Cutoff

Introducing a threshold $m_0 \in [16, 32]$ for handing off small subproblems to *Insertion Sort* directly leverages hardware caching characteristics and execution profile:

- **Elimination of Recursive Overhead:** When subarray size drops below m_0 , the overhead of function call frames, parameter passing, and partition pointer management surpasses the computational cost of direct comparisons.
- **Low Constant Factors of Insertion Sort:** *Insertion Sort* has exceptionally low constant factors: it requires no extra heap or stack space, runs tight inner loops, and easily undergoes compiler vectorization and loop unrolling.

- **Optimal L1 Cache Locality:** A block of $16 \div 32$ items requires approximately $128 \div 256$ bytes (for 64-bit keys), fitting completely within one or two 64-byte L1 cache lines. All comparisons and shifts occur without generating any cache misses.
- **Nearly Sorted Behavior:** Because Quicksort's macro-partitions have already placed elements near their final positions, small subarrays are almost sorted. On nearly sorted inputs, Insertion Sort operates in linear time $\mathcal{O}(n)$ rather than quadratic time.

In conclusion, Bounded Quicksort represents an ideal synthesis of mathematical rigor and algorithm engineering, guaranteeing optimal expected time complexity $\mathcal{O}(n \log_2 n)$, absolute immunity to *Stack Overflow* via $\Theta(\log_2 n)$ auxiliary space, and exceptional empirical throughput on modern computer architectures.

Bibliography

- [1] Donald E. Knuth. *The Art of Computer Programming*, volume 1. Addison-Wesley, Reading, MA, 3 edition, 1997.
- [2] Alok Aggarwal and Jeffrey S. Vitter. The input/output complexity of sorting and related problems. *Communications of the ACM*, 31(9):1116–1127, 1988.
- [3] Edsger W. Dijkstra. *A Discipline of Programming*. Prentice-Hall, 1976.
- [4] Robert Sedgewick. Implementing quicksort programs. *Communications of the ACM*, 21(10):847–857, 1978.
- [5] Vladimir Yaroslavskiy. Dual-pivot quicksort algorithm. *Research Disclosure*, 2009. Available at <http://iaroslavski.narod.ru/quicksort/DualPivotQuicksort.pdf>.